HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA AN TOÀN THÔNG TIN

BÁO CÁO
HỌC PHẦN: IOT VÀ ỨNG DỤNG
MÃ HỌC PHẦN: INT14149

ĐỀ TÀI: HỆ THỐNG CẢM BIẾN GIÁM SÁT MÔI TRƯỜNG VÀ ĐIỀU KHIỂN THIẾT BỊ

B23DCAT271 - Triệu Nguyễn Minh Tân

HÀ NỘI 2026

MỤC LỤC

MỤC LỤC 2
CHƯƠNG 1. GIỚI THIỆU 3
1.1 Mục đích 3
1.2 Phạm vi hệ thống 3
CHƯƠNG 2. MÔ TẢ TỔNG QUAN 5
2.1 Bối cảnh và kiến trúc hệ thống 5
2.2 Chức năng chính của hệ thống 5
2.3 Công nghệ sử dụng 6
2.4. Giả định và ràng buộc chung 8
CHƯƠNG 3. USE CASE 10
3.1 Danh sách một số Use Case 10
3.2 Chi tiết một số Use Case 10
CHƯƠNG 4. KẾT QUẢ THỰC NGHIỆM 27
4.1 Môi trường và kịch bản thực nghiệm 27
4.2 Kết quả triển khai phần cứng và thu thập dữ liệu cảm biến 28
4.3 Kết quả điều khiển thiết bị và đánh giá độ trễ phản hồi 32
4.4 Kết quả tra cứu, lọc và phân trang dữ liệu 35
4.5 Kết quả phân quyền người dùng (RBAC) và kiểm soát bảo mật 37
4.6 Kết quả kiểm thử tự động hóa bộ API trên Postman 39
4.7 Kết quả thực nghiệm khi gặp sự cố khi mất kết nối NodeMCU 41
4.8 Đánh giá chất lượng và các cơ chế chịu lỗi của hệ thống 44
CHƯƠNG 5. TỔNG KẾT 45
5.1 Kết quả đạt được của đề tài 45
5.2 Những hạn chế và bài học kinh nghiệm 47
5.3 Hướng phát triển trong tương lai 48

CHƯƠNG 1. GIỚI THIỆU

1.1 Mục đích

Tài liệu này đặc tả toàn diện các yêu cầu chức năng (Functional Requirements) và yêu cầu phi chức năng (Non-Functional Requirements) của đề tài "Hệ thống cảm biến giám sát môi trường và điều khiển thiết bị" — một giải pháp IoT hoàn chỉnh phục vụ nghiên cứu và thực hành chuyên sâu trong học phần IoT và Ứng dụng (Mã học phần: INT14149) tại Học viện Công nghệ Bưu chính Viễn thông.

Tài liệu đóng vai trò là bản đặc tả kỹ thuật phần mềm và hệ thống (Software Requirements Specification - SRS), thiết lập cơ sở kỹ thuật thống nhất giữa sinh viên thực hiện, giảng viên hướng dẫn và hội đồng đánh giá trong việc:

- Định hình kiến trúc tổng thể từ phần cứng biên (Edge IoT Hardware), mạng truyền thông (MQTT), máy chủ điều phối trung tâm (Node-RED Backend), cơ sở dữ liệu quan hệ (MySQL 8.0) đến giao diện trực quan hóa dữ liệu (Web Dashboard SPA).
- Xác định phạm vi tính năng, mô hình dữ liệu, cơ chế phân quyền người dùng (Role-Based Access Control) và các kịch bản kiểm thử tự động (Automated Testing).
- Làm căn cứ đối chiếu, nghiệm thu chất lượng mã nguồn firmware, luồng xử lý backend, thiết kế cơ sở dữ liệu và sản phẩm giao diện web.

1.2 Phạm vi hệ thống

Hệ thống được thiết kế và triển khai nhằm phục vụ bài toán giám sát điều kiện môi trường thực tế kết hợp với điều khiển thiết bị chấp hành hai chiều có phản hồi xác nhận. Phạm vi triển khai của hệ thống bao gồm:

- Thu thập dữ liệu môi trường thời gian thực: Đo lường định kỳ 3 thông số môi trường cốt lõi bao gồm Nhiệt độ không khí (°C), Độ ẩm không khí (%) thông qua cảm biến kỹ thuật số DHT11 (kết nối chân D2/GPIO4) và Cường độ ánh sáng môi trường (lux) thông qua module quang trở LDR (kết nối chân analog A0/ADC0) trên vi điều khiển NodeMCU v3 (ESP8266).
- Truyền dẫn dữ liệu qua giao thức MQTT: Thiết bị biên đóng gói dữ liệu đo đạc thành định dạng chuẩn JSON ({"temp": float, "hum": float, "light": int}) và xuất bản (Publish) định kỳ 5 giây/lần lên topic data/sensor tới MQTT Broker cục bộ Eclipse Mosquitto (cổng 8000).
- Điều khiển thiết bị chấp hành 2 chiều (Bidirectional Control): Cho phép điều khiển độc lập hoặc đồng thời 2 đèn LED công suất nhỏ (LED 1 nối chân D5/GPIO14, LED 2 nối chân D6/GPIO12 qua điện trở hạn dòng 220Ω). Lệnh điều khiển từ Web Dashboard hoặc REST API được gửi qua các topic MQTT (device/control/led1, device/control/led2, device/control/all). Mạch vi điều khiển thực thi và gửi phản hồi xác nhận (ACK) về topic device/response để máy chủ ghi nhận kết quả và tính toán độ trễ mạng (response_ms).

- Lưu trữ và toàn vẹn dữ liệu quan hệ: Toàn bộ dữ liệu vi khí hậu (Telemetry) và nhật ký thao tác điều khiển được lưu trữ trong cơ sở dữ liệu MySQL 8.0 gồm 5 bảng chuẩn theo ERD (users, devices, actions, sensors, data_sensors).
- Xác thực và phân quyền người dùng (RBAC): Hệ thống quản lý phiên đăng nhập qua Session Token với 3 phân quyền nghiêm ngặt:
  - Quản trị viên (Admin): Toàn quyền điều khiển thiết bị, xem toàn bộ màn hình và cấu hình hệ thống.
  - Người dùng (User): Toàn quyền bật/tắt thiết bị, xem Dashboard, lịch sử cảm biến, nhật ký hành động và thông tin cá nhân.
  - Người xem (Viewer): Chỉ được phép quan sát dữ liệu trên Dashboard; bị chặn tuyệt đối quyền bấm công tắc điều khiển và không được phép truy cập các màn hình quản trị dữ liệu.
- Giao diện Web Dashboard thời gian thực: Ứng dụng Single Page Application (SPA) phát triển trên nền HTML5, CSS3, JavaScript ES6 và Tailwind CSS, tích hợp thư viện Chart.js để vẽ biểu đồ diễn biến 3 đường cảm biến động, hỗ trợ tìm kiếm, lọc theo loại cảm biến, phân trang linh hoạt, tự động phát hiện mất kết nối phần cứng NodeMCU sau ~7 giây và duy trì trạng thái khi tải lại trang (Persistence via LocalStorage).

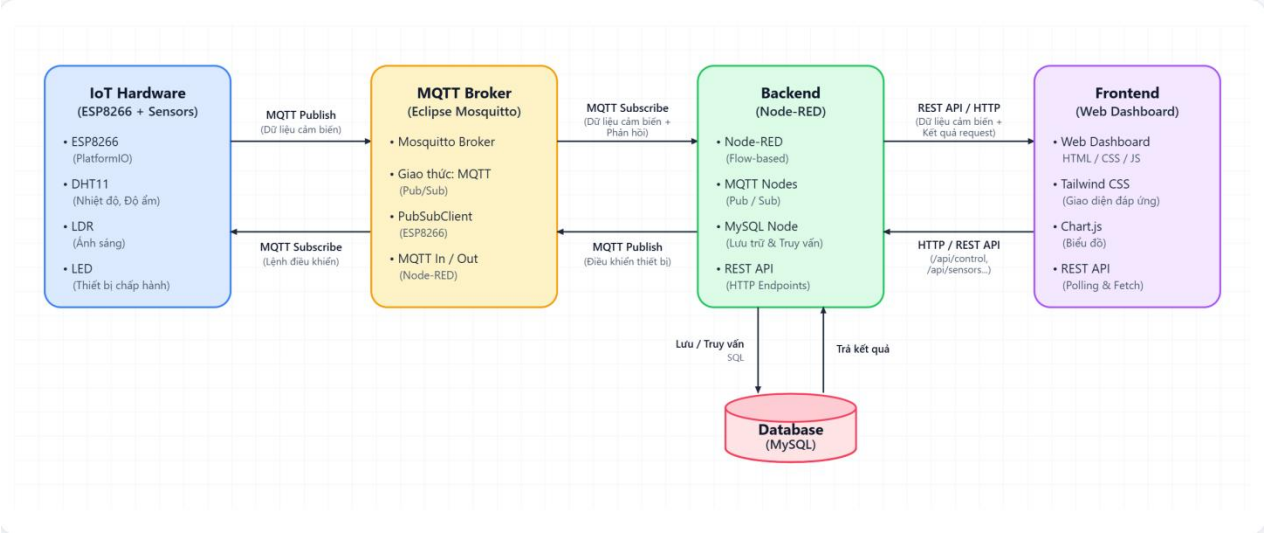
Phạm vi giới hạn: Đề tài tập trung vào việc làm chủ kiến trúc phần mềm, giao thức IoT, thiết kế CSDL và giao diện ứng dụng. Đề tài không bao gồm việc gia công khuôn vỏ công nghiệp hàng loạt, không sử dụng các dịch vụ đám mây thương mại đóng kín (như Blynk, ThingSpeak, Adafruit IO) mà hoàn toàn tự triển khai và làm chủ hệ thống trên hạ tầng máy chủ cục bộ (On-premises / Local Server).

CHƯƠNG 2. MÔ TẢ TỔNG QUAN

2.1 Bối cảnh và kiến trúc hệ thống

Hệ thống được thiết kế theo mô hình kiến trúc phân tầng chuyên biệt (Multi-tier IoT Architecture) kết hợp luồng xử lý định hướng sự kiện (Event-driven Architecture), đảm bảo tính tách biệt trách nhiệm (Separation of Concerns), độ tin cậy và khả năng mở rộng cao.

Kiến trúc tổng thể của hệ thống:



Hình 1 - Kiến trúc tổng thể hệ thống

Tóm tắt 2 luồng dữ liệu chính trong hệ thống:

- Luồng thu thập dữ liệu (Telemetry Stream): Cảm biến → NodeMCU ESP8266 → MQTT Broker (data/sensor) → Node-RED → Bảng data_sensors (MySQL) → Web Dashboard (Polling chu kỳ 2 giây qua API /api/dashboard).
- Luồng điều khiển thiết bị (Control Stream): Thao tác người dùng trên Web / Gọi REST API → Node-RED (POST /api/control) → Kiểm tra quyền → Ghi nhật ký trạng thái pending vào bảng actions → MQTT Broker (device/control/*) → NodeMCU thực thi ngắt/bật chân GPIO → NodeMCU gửi phản hồi về Broker (device/response) → Node-RED đối soát mã lệnh → Cập nhật trạng thái success và ghi nhận response_ms vào bảng actions → Cập nhật tức thời trạng thái công tắc trên giao diện web.

2.2 Chức năng chính của hệ thống

- Giám sát vi khí hậu thời gian thực:
- Điều khiển thiết bị chấp hành linh hoạt:
- Đo lường độ trễ mạng và Kiểm toán hành động (Audit Trail):
- Phân quyền người dùng theo vai trò (Role-Based Access Control - 3 Roles)

- Tra cứu, tìm kiếm, lọc và phân trang dữ liệu lớn:
- Tự động giám sát phần cứng & Cảnh báo mất kết nối (Offline Detection)
- Đồng bộ hóa và Khôi phục trạng thái khi tải lại trang (State Persistence)

2.3 Công nghệ sử dụng

2.3.1 Lớp thiết bị (Device Layer)

- Phần cứng vi điều khiển: Bo mạch NodeMCU v3 kit (ESP-12E Module tích hợp vi xử lý 32-bit Tensilica L106, xung nhịp 80MHz, bộ nhớ Flash 4MB, chip nạp cổng nối tiếp CH340).
- Môi trường phát triển firmware: PlatformIO IDE tích hợp trong Visual Studio Code / Antigravity IDE, sử dụng gói framework Arduino core for ESP8266.
- Cảm biến và cơ cấu chấp hành:
  - Cảm biến nhiệt độ và độ ẩm kỹ thuật số DHT11 (kết nối chân dữ liệu vào GPIO4 / D2).
  - Cảm biến ánh sáng sử dụng Quang trở LDR mắc theo mạch phân áp (kết nối chân ngõ ra tương tự vào chân analog A0 / ADC0).
  - 02 Đèn LED siêu sáng 5mm (Đèn 1 nối chân GPIO14 / D5; Đèn 2 nối chân GPIO12 / D6) qua 2 điện trở hạn dòng 220Ω.
- Các thư viện mã nguồn nhúng:
  - ESP8266WiFi.h: Quản lý kết nối mạng Wi-Fi và lớp mạng TCP/IP.
  - PubSubClient.h (tác giả Nick O'Leary): Đóng vai trò MQTT Client gọn nhẹ, cấu hình kích thước bộ đệm MQTT_MAX_PACKET_SIZE = 256 bytes và client.setKeepAlive(5).
  - ArduinoJson.h (phiên bản v6.x): Đóng gói và bóc tách cấu trúc dữ liệu JSON tốc độ cao trực tiếp trên bộ nhớ tĩnh RAM (StaticJsonDocument).
  - DHT.h (Adafruit): Đọc và chuẩn hóa dữ liệu từ cảm biến DHT11.

2.3.2 Lớp kết nối/truyền thông (Network Layer)

- Giao thức truyền thông chính: MQTT v3.1.1 (Message Queuing Telemetry Transport) chạy trên nền tảng mạng TCP/IP.
- Máy chủ môi giới (MQTT Broker): Eclipse Mosquitto v2.x, triển khai dưới dạng Windows Service cục bộ, lắng nghe kết nối tại cổng TCP 8000.
- Hạ tầng mạng kết nối vật lý: Mạng Wi-Fi cục bộ chuẩn IEEE 802.11 b/g/n băng tần 2.4 GHz.
- Đặc tả danh mục Topic MQTT trong hệ thống:
  - data/sensor (ESP8266 Publish → Node-RED Subscribe): Dữ liệu cảm biến định kỳ 5s/lần

- device/control/led1, device/control/led2 (Node-RED Publish → ESP8266 Subscribe): Lệnh điều khiển đơn lẻ từng đèn với payload chuỗi: "ON" hoặc "OFF".
- device/control/all (Node-RED Publish → ESP8266 Subscribe): Lệnh điều khiển đồng thời cả 2 đèn
- device/response (ESP8266 Publish → Node-RED Subscribe): Phản hồi kết quả thực thi phần cứng với payload:

{"id": 1, "device": "led1", "state": "ON", "status": "success"}

2.3.3 Lớp ứng dụng (Application Layer)

- Máy chủ dịch vụ Backend: Node-RED Runtime v3.x (vận hành trên nền Node.js LTS, cổng dịch vụ 8800).
  - Thực hiện toàn bộ chức năng của một Backend hoàn chỉnh: đón nhận bản tin MQTT, chuyển tiếp dữ liệu, xử lý thuật toán phân quyền, sinh session token, đo đạc thời gian phản hồi và cung cấp bộ RESTful API.
  - Node giao tiếp: mqtt in, mqtt out, http in, http response, function, node-red-node-mysql.
- Hệ quản trị cơ sở dữ liệu: MySQL Community Server 8.0 (cổng dịch vụ 3306), lưu trữ CSDL iot_project với bảng mã utf8mb4_unicode_ci và engine lưu trữ InnoDB. Cấu trúc 5 bảng chuẩn ERD:
  - users: Lưu trữ thông tin tài khoản, mật khẩu và vai trò (admin, user, viewer).
  - devices: Danh mục thiết bị điều khiển (led1, led2).
  - actions: Nhật ký điều khiển, kết nối khóa ngoại với devices và users, lưu trữ độ trễ response_ms.
  - sensors: Danh mục cảm biến (dht11_temp, dht11_hum, ldr_light).
  - data_sensors: Lưu trữ số liệu đo đạc thực tế của cảm biến theo mốc thời gian.
- Giao diện Web Client (Frontend Dashboard SPA):
  - Kiến trúc: Single Page Application hiện đại, không phụ thuộc framework cồng kềnh, tối ưu hiệu năng cao.
  - Công nghệ xây dựng: HTML5 chuẩn Semantic, CSS3 hiện đại, JavaScript thuần (Vanilla ES6+).
  - Thư viện giao diện: Tailwind CSS (Utility-first CSS Framework) kết hợp bộ icon vector Lucide Icons.
  - Trực quan hóa dữ liệu: Chart.js v4.x hỗ trợ vẽ biểu đồ đường đa trục (Multi-axis Line Chart) mượt mà.
  - Đồng bộ hóa: Kỹ thuật HTTP REST Polling chu kỳ ngắn (2 giây) đảm bảo dữ liệu hiển thị tức thời mà không gây quá tải tài nguyên máy chủ.

- Công cụ thiết kế & Kiểm thử:
  - Figma: Thiết kế wireframe, bố cục giao diện UI/UX của 4 màn hình chính trước khi chuyển giao sang mã nguồn HTML.
  - Postman: Bộ kiểm thử API hoàn chỉnh (Collection & Environment) tự động hóa kiểm tra tính đúng đắn 31 Requests và 66 Test Assertions (bao gồm cả phân hệ kiểm thử phân quyền RBAC và token).

2.4. Giả định và ràng buộc chung

Trong quá trình thiết kế, triển khai và vận hành hệ thống, các giả định và ràng buộc kỹ thuật sau được áp dụng chặt chẽ:

- Ràng buộc hạ tầng mạng không dây (Wi-Fi 2.4GHz):
  - Vi điều khiển ESP8266 chỉ hỗ trợ băng tần Wi-Fi 2.4 GHz (chuẩn 802.11 b/g/n), không tương thích với băng tần 5 GHz. Do đó, điểm phát sóng Wi-Fi thử nghiệm bắt buộc phải kích hoạt băng tần 2.4 GHz.
  - Máy tính chạy Node-RED, Mosquitto Broker và bo mạch NodeMCU phải kết nối chung trong một dải mạng con (Subnet IP) để đảm bảo các gói tin TCP/IP và bản tin MQTT có thể định tuyến thông suốt.
- Ràng buộc phần cứng vi điều khiển & Cảm biến:
  - Vi điều khiển NodeMCU chỉ tích hợp duy nhất 01 kênh analog ADC (chân A0) với dải điện áp đo từ 0V đến 1.0V (bên trong module NodeMCU đã có mạch phân áp mở rộng dải đo lên 0V−3.3V). Thang đo số nhận được nằm trong khoảng 0−10230 - 10230−1023.
  - Cảm biến DHT11 có giới hạn phần cứng về thời gian đáp ứng: không được phép đọc dữ liệu cảm biến với tần số nhanh hơn 1 – 2 giây/lần. Hệ thống lựa chọn chu kỳ đọc 5 giây/lần là hoàn toàn tối ưu và bảo vệ độ bền cho cảm biến.
  - Dòng điện định mức của các chân GPIO ESP8266 tối đa khoảng 12mA. Khi nối 2 đèn LED vào chân D5 và D6, bắt buộc phải mắc nối tiếp điện trở hạn dòng 220Ω để chống hiện tượng sụt áp và bảo vệ chip.
- Ràng buộc môi trường biên dịch phần mềm:

Để tránh lỗi trình biên dịch Xtensa GCC (Invalid argument), thư mục dự án của mã nguồn PlatformIO bắt buộc phải đặt tại thư mục không chứa dấu tiếng Việt và không chứa khoảng trắng (khuyến nghị tại D:\iot-project hoặc C:\iot-project).

- Ràng buộc cơ sở dữ liệu và toàn vẹn dữ liệu:
  - Toàn bộ cơ sở dữ liệu phải tuân thủ nghiêm ngặt chuẩn quan hệ 5 bảng theo ERD đã được phê duyệt. Mọi hành động điều khiển chỉ được ghi nhận thành công khi đáp ứng ràng buộc khóa ngoại tham chiếu tới device_id và user_id hợp lệ.

- Lệnh điều khiển gửi tới thiết bị nếu sau 5000ms không nhận được phản hồi từ NodeMCU sẽ được máy chủ tự động chuyển trạng thái sang timeout.

- Nguồn cấp điện:

Hệ thống phiên bản phòng thí nghiệm được cấp nguồn ổn định thông qua cáp micro-USB 5V cắm trực tiếp từ cổng USB máy tính (cổng COM6 nối tiếp) hoặc củ sạc điện thoại 5V−1A, không sử dụng nguồn pin dự phòng trong giai đoạn thử nghiệm này.

CHƯƠNG 3. USE CASE

3.1 Danh sách một số Use Case

Hệ thống giám sát và điều khiển IoT phục vụ 3 nhóm đối tượng người dùng tương ứng với mô hình phân quyền Role-Based Access Control (RBAC): Quản trị viên (Admin), Người dùng tiêu chuẩn (User) và Khách vãng lai (Viewer), cùng với tác nhân hệ thống là vi điều khiển NodeMCU ESP8266.

Bảng 1. Bảng tổng hợp danh sách các Use Case chính của hệ thống:

Mã Use Case	Tên Use Case	Tác nhân chính (Actor)	Mô tả tóm tắt chức năng
UC-01	Xem dữ liệu môi trường thời gian thực	Admin, User, Viewer	Mở Dashboard theo dõi nhiệt độ (°C), độ ẩm (%), cường độ ánh sáng (lux) và biểu đồ xu hướng biến thiên thời gian thực (chu kỳ gửi 5s/lần từ NodeMCU qua MQTT).
UC-02	Điều khiển đèn (LED 1 & LED 2)	Admin, User	Bấm công tắc trên Dashboard hoặc gọi REST API để bật/tắt Đèn 1 (D5) và Đèn 2 (D6) đơn lẻ hoặc đồng thời; nhận phản hồi xác nhận trạng thái và đo thời gian phản hồi (ms).
UC-03	Đăng nhập & Xác thực hệ thống	Tất cả người dùng	Đăng nhập tài khoản bằng Username/Password; hệ thống xác thực từ bảng `users`, sinh phiên làm việc (Session Token) và cấp quyền truy cập giao diện tương ứng với vai trò.
UC-04	Thông báo lỗi khi mất kết nối NodeMCU	Hệ thống (Node-RED, Dashboard)	Hệ thống tự động phát hiện khi quá 10 giây không nhận được gói tin MQTT từ NodeMCU; hiển thị thanh cảnh báo đỏ (DISCONNECTED) và tự tắt khi thiết bị kết nối lại.
UC-05	Khôi phục dữ liệu và trạng thái khi tải lại trang	Người dùng cuối	Khi người dùng nhấn F5 hoặc tải lại trang, hệ thống tự động truy vấn MySQL khôi phục chính xác trạng thái thực tế của 2 đèn, số liệu cảm biến mới nhất và 20 điểm biểu đồ gần nhất.

3.2 Chi tiết một số Use Case

3.2.1 UC-01 – Xem dữ liệu môi trường thời gian thực

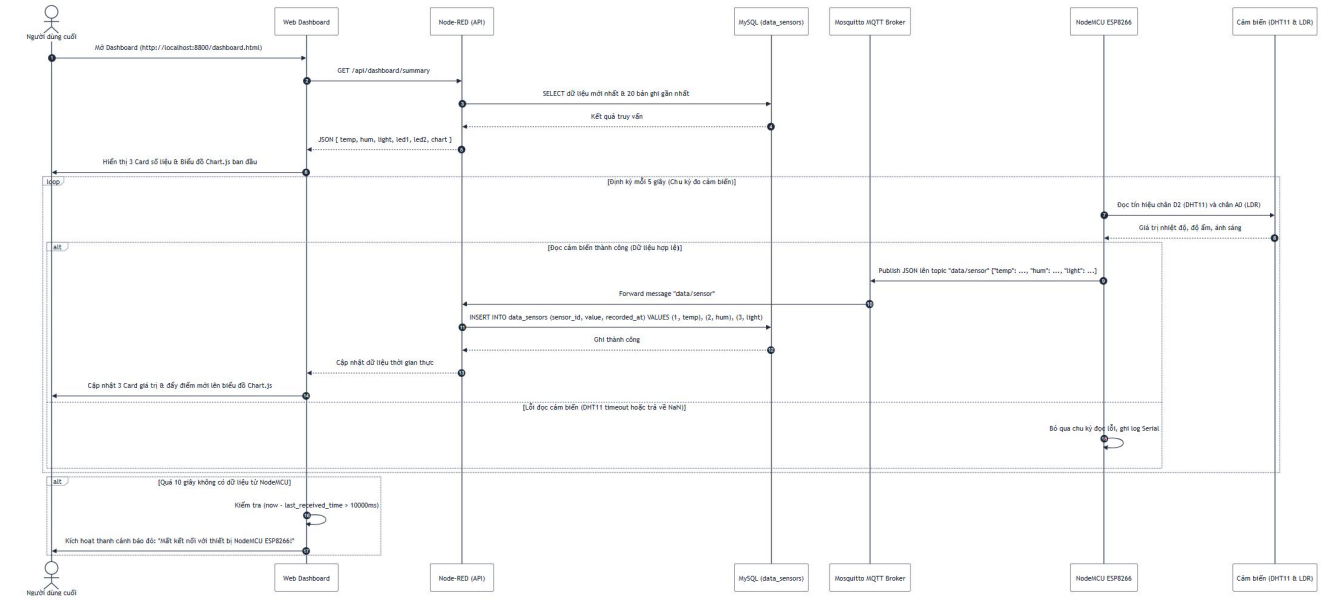
3.2.1.1 Bảng đặc tả

Mục	Nội dung chi tiết
Tên Use Case	UC-01 – Xem dữ liệu môi trường thời gian thực
Actor	- Actor chính (Primary): Người dùng cuối (Admin, User, Viewer)

Mục	Nội dung chi tiết
	- Actor phụ (Supporting): NodeMCU ESP8266 (thiết bị đo), Mosquitto MQTT Broker (trung gian truyền thông), Node-RED (xử lý & ghi nhận cơ sở dữ liệu), Cơ sở dữ liệu MySQL (iot_project).
Mô tả tóm tắt	Người dùng truy cập màn hình Dashboard trên trình duyệt web để theo dõi các giá trị đo môi trường gồm: Nhiệt độ (°C), Độ ẩm (%) từ cảm biến DHT11 và Cường độ ánh sáng (lux) từ quang trở LDR. Dữ liệu được đo đạc định kỳ mỗi 5 giây bởi NodeMCU, truyền tải qua giao thức MQTT, ghi lưu vào cơ sở dữ liệu MySQL và cập nhật trực tiếp lên các thẻ hiển thị (Cards) cũng như biểu đồ đường (Chart.js).
Tiền điều kiện	1. NodeMCU ESP8266 đã kết nối thành công vào mạng WiFi cục bộ và thiết lập kết nối tới Mosquitto MQTT Broker (port 8000/1883). 2. Cảm biến DHT11 (chân D2/GPIO4) và Quang trở LDR (chân A0) được cấp nguồn và hoạt động ổn định. 3. Node-RED đã khởi chạy, kết nối tới Mosquitto Broker và đã subscribe topic data/sensor. 4. Người dùng mở trang Dashboard (http://localhost:8800/dashboard.html) trên trình duyệt web.
Hậu điều kiện	1. Giá trị Nhiệt độ, Độ ẩm, Ánh sáng mới nhất hiển thị chính xác trên 3 thẻ Card giám sát của Dashboard. 2. Biểu đồ Chart.js tự động vẽ thêm điểm dữ liệu mới nhất (giữ tối đa 20 mốc gần nhất theo thời gian thực). 3. Bản ghi đo đạc được lưu trữ thành công vào bảng data_sensors trong cơ sở dữ liệu MySQL iot_project (tương ứng 3 bản ghi cho 3 sensor_id: 1 - nhiệt độ, 2 - độ ẩm, 3 - ánh sáng). 4. Thanh cảnh báo mất kết nối NodeMCU (nếu có trước đó) được tự động ẩn đi.
Kịch bản chính (Main Flow)	1. Người dùng mở trình duyệt web truy cập địa chỉ Dashboard. 2. Trình duyệt gửi yêu cầu GET /api/dashboard tới Node-RED Backend để nạp trạng thái mới nhất và 20 bản ghi lịch sử phục vụ vẽ biểu đồ ban đầu. 3. Định kỳ mỗi 5 giây, vi điều khiển NodeMCU đọc giá trị vật lý từ cảm biến DHT11 (Nhiệt độ, Độ ẩm) và ADC chân A0 (Quang trở LDR). 4. NodeMCU đóng gói dữ liệu thành chuỗi JSON: {"temp": float, "hum": float, "light": int} và publish lên MQTT Broker tại topic data/sensor với QoS 0. 5. Mosquitto MQTT Broker nhận bản tin và chuyển tiếp ngay tới Node-RED. 6. Node-RED nhận message, phân tích (parse) chuỗi JSON và kiểm tra tính hợp lệ của dữ liệu.

Mục	Nội dung chi tiết
	7. Node-RED thực thi câu lệnh SQL INSERT 3 dòng tương ứng vào bảng data_sensors trong MySQL kèm mốc thời gian recorded_at. 8. Web Dashboard thông qua cơ chế Polling (chu kỳ 2 giây) hoặc nhận dữ liệu mới qua API, cập nhật ngay lập tức các chỉ số lên 3 thẻ Card và đẩy điểm mới vào biểu đồ Chart.js. 9. Người dùng quan sát sự biến thiên của các thông số môi trường trực quan trên màn hình.
Ngoại lệ (Exception Flows)	E1 - Lỗi đọc cảm biến DHT11 (Timeout hoặc trả về NaN): NodeMCU kiểm tra thấy giá trị không hợp lệ.

3.2.1.2 Sequence Diagram



Hình 2 - Sơ đồ tuần tự (Sequence Diagram) của UC-01

1. Người dùng mở trình duyệt web truy cập địa chỉ Dashboard.
2. Trình duyệt gửi yêu cầu GET /api/dashboard tới Node-RED Backend để nạp trạng thái mới nhất và 20 bản ghi lịch sử phục vụ vẽ biểu đồ ban đầu.
3. Định kỳ mỗi 5 giây, vi điều khiển NodeMCU đọc giá trị vật lý từ cảm biến DHT11 (Nhiệt độ, Độ ẩm) và ADC chân A0 (Quang trở LDR).
4. NodeMCU đóng gói dữ liệu thành chuỗi JSON: {"temp": float, "hum": float, "light": int} và publish lên MQTT Broker tại topic data/sensor với QoS 0.
5. Mosquitto MQTT Broker nhận bản tin và chuyển tiếp ngay tới Node-RED.

6. Node-RED nhận message, phân tích (parse) chuỗi JSON và kiểm tra tính hợp lệ của dữ liệu.
7. Node-RED thực thi câu lệnh SQL INSERT 3 dòng tương ứng vào bảng data_sensors trong MySQL kèm mốc thời gian recorded_at.
8. Web Dashboard thông qua cơ chế Polling (chu kỳ 2 giây) hoặc nhận dữ liệu mới qua API, cập nhật ngay lập tức các chỉ số lên 3 thẻ Card và đẩy điểm mới vào biểu đồ Chart.js.
9. Người dùng quan sát sự biến thiên của các thông số môi trường trực quan trên màn hình.

3.2.2 UC-02 – Điều khiển đèn

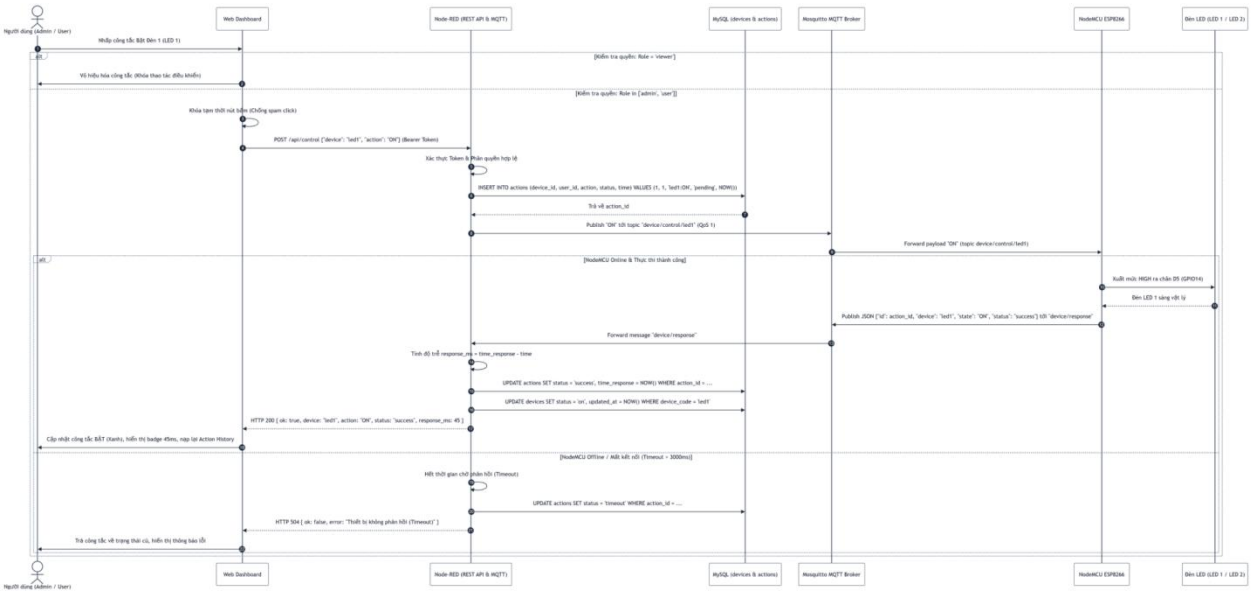
3.2.2.1 Bảng đặc tả

Mục	**Nội dung chi tiết**
Tên Use Case	UC-02 – Điều khiển thiết bị đèn (LED 1 và LED 2)
Actor	- Actor chính (Primary): Người dùng (Admin hoặc User) - Actor phụ (Supporting): Web Dashboard, Node-RED Backend (REST API), Mosquitto MQTT Broker, NodeMCU ESP8266, Thiết bị chấp hành (LED 1 chân D5, LED 2 chân D6), Cơ sở dữ liệu MySQL.
Mô tả tóm tắt	Người dùng thực hiện thao tác bật hoặc tắt Đèn 1 (LED 1 nối chân D5 qua trở 220Ω) và Đèn 2 (LED 2 nối chân D6 qua trở 220Ω) độc lập hoặc đồng thời từ xa thông qua công tắc trên giao diện Web Dashboard hoặc qua Postman REST API. Lệnh điều khiển được gửi tới Backend, ghi nhận lịch sử vào bảng actions với trạng thái pending, truyền qua MQTT tới NodeMCU để đóng/ngắt chân GPIO. Sau khi thực thi phần cứng, NodeMCU gửi phản hồi trạng thái xác nhận về topic device/response. Backend tính toán thời gian đáp ứng (response_ms), cập nhật trạng thái success/failed vào cơ sở dữ liệu và đồng bộ hiển thị lên giao diện người dùng.
Tiền điều kiện	1. Người dùng đã đăng nhập hệ thống với tài khoản có vai trò hợp lệ: Admin (admin/admin123) hoặc User (user/user123). *(Tài khoản viewer bị vô hiệu hóa công tắc điều khiển)*. 2. NodeMCU ESP8266 đang hoạt động (Online), đã kết nối WiFi và subscribe các topic điều khiển: device/control/led1, device/control/led2, device/control/all. 3. Mosquitto MQTT Broker và Node-RED hoạt động bình thường trên cổng quy định (8000 và 8800).
Hậu điều	1. Đèn LED vật lý (LED 1 / LED 2) chuyển sang trạng thái sáng (HIGH)

Mục	Nội dung chi tiết
kiện	hoặc tắt (LOW) chính xác theo lệnh. 2. Trạng thái hoạt động của thiết bị trong bảng devices được cập nhật thành on hoặc off. 3. Một bản ghi lịch sử điều khiển hoàn chỉnh được lưu trong bảng actions với các trường: device_id, user_id, action (VD: led1:ON), status (success), time, time_response, và độ trễ đáp ứng response_ms. 4. Công tắc trên Dashboard chuyển sang màu và vị trí mới (Xanh lá khi BẬT, Xám khi TẮT) kèm badge hiển thị thời gian phản hồi (ms), đồng thời thêm ngay một dòng vào bảng Action History.
Kịch bản chính (Main Flow)	1. Người dùng quan sát trạng thái công tắc hiện tại của Đèn 1 hoặc Đèn 2 trên Dashboard. 2. Người dùng nhấp chuột vào công tắc để bật hoặc tắt đèn (hoặc chọn điều khiển đồng thời cả 2 đèn). 3. Giao diện Dashboard gửi yêu cầu HTTP POST /api/control (kèm header Authorization Bearer Token) với body JSON: {"device": "led1", "action": "ON"} tới Node-RED Backend. 4. Node-RED kiểm tra tính hợp lệ của Token và xác định quyền của tài khoản (Admin hoặc User). 5. Node-RED thêm một bản ghi mới vào bảng actions trong MySQL với trạng thái pending và ghi nhận mốc thời gian gửi lệnh time. 6. Node-RED publish payload ON lên MQTT topic device/control/led1 với QoS 1. 7. Mosquitto MQTT Broker nhận lệnh và chuyển tiếp ngay tới NodeMCU ESP8266. 8. NodeMCU nhận gói tin, phân tích mã thiết bị và hành động; xuất mức tín hiệu điện áp tương ứng ra chân GPIO14/D5 (digitalWrite(LED1_PIN, HIGH)). 9. Đèn LED 1 vật lý bật sáng ngay lập tức. 10. NodeMCU đóng gói bản tin JSON xác nhận: {"id": action_id, "device": "led1", "state": "ON", "status": "success"} và publish lên topic device/response. 11. Mosquitto Broker chuyển tiếp bản tin phản hồi về Node-RED. 12. Node-RED nhận phản hồi, tính độ trễ: response_ms = time_response - time, sau đó cập nhật bản ghi trong bảng actions thành status = 'success', đồng thời cập nhật trạng thái status = 'on' trong bảng devices. 13. Node-RED trả về phản hồi HTTP 200 cho Web Dashboard: {"ok": true, "device": "led1", "action": "ON", "status": "success", "response_ms": 45}. 14. Dashboard chuyển trạng thái công tắc sang ON, hiển thị huy hiệu phản hồi (ví dụ: 45ms) và tự động nạp lại bảng Action History nếu đang mở. 15. Người dùng quan sát thấy đèn LED vật lý bật sáng và giao diện hiển thị

Mục	Nội dung chi tiết
	đồng bộ.
Ngoại lệ (Exception Flows)	• E1 - Phân quyền từ chối (Tài khoản Viewer): Nếu người dùng có vai trò viewer, Dashboard khóa hoàn toàn công tắc (vô hiệu hóa nhấp chuột). Nếu gửi request trực tiếp qua Postman, API trả về HTTP 403 Forbidden: {"ok": false, "error": "Tài khoản viewer chỉ có quyền xem, không được điều khiển thiết bị"}. • E2 - NodeMCU mất kết nối hoặc không phản hồi (Timeout): Sau 5000ms kể từ khi phát lệnh mà Node-RED không nhận được message phản hồi trên topic device/response, Node-RED tự động cập nhật bản ghi trong bảng actions thành status = 'timeout' và trả về Timeout cho Dashboard để người dùng biết lệnh chưa thực thi thành công. • E3 - Mã thiết bị hoặc hành động không hợp lệ: Nếu body request chứa thiết bị khác ngoài led1, led2 hoặc hành động khác ON, OFF, Node-RED lập tức trả về HTTP 400 Bad Request: {"ok": false, "error": "Thiết bị không hợp lệ (chỉ hỗ trợ led1, led2)"} mà không gửi gói tin ra MQTT Broker. • E4 - Thao tác nhấp chuột liên tục (Debounce): Khi người dùng nhấp công tắc, giao diện tạm thời làm mờ (disable) công tắc trong 500ms cho đến khi nhận được phản hồi từ Backend, ngăn chặn tình trạng gửi dồn dập nhiều lệnh gây nghẽn hàng đợi MQTT.

3.2.2.2 Sequence Diagram



Hình 3 - Sơ đồ tuần tự (Sequence Diagram) của UC-02

1. Người dùng quan sát trạng thái công tắc hiện tại của Đèn 1 hoặc Đèn 2 trên Dashboard.
2. Người dùng nhấp chuột vào công tắc để bật hoặc tắt đèn (hoặc chọn điều khiển đồng thời cả 2 đèn).
3. Giao diện Dashboard gửi yêu cầu HTTP POST /api/control (kèm header Authorization Bearer Token) với body JSON: {"device": "led1", "action": "ON"} tới Node-RED Backend.
4. Node-RED kiểm tra tính hợp lệ của Token và xác định quyền của tài khoản (Admin hoặc User).
5. Node-RED thêm một bản ghi mới vào bảng actions trong MySQL với trạng thái pending và ghi nhận mốc thời gian gửi lệnh time.
6. Node-RED publish payload ON lên MQTT topic device/control/led1 với QoS 1.
7. Mosquitto MQTT Broker nhận lệnh và chuyển tiếp ngay tới NodeMCU ESP8266.
8. NodeMCU nhận gói tin, phân tích mã thiết bị và hành động; xuất mức tín hiệu điện áp tương ứng ra chân GPIO14/D5 (digitalWrite(LED1_PIN, HIGH)).
9. Đèn LED 1 vật lý bật sáng ngay lập tức.
10. NodeMCU đóng gói bản tin JSON xác nhận: {"id": action_id, "device": "led1", "state": "ON", "status": "success"} và publish lên topic device/response.
11. Mosquitto Broker chuyển tiếp bản tin phản hồi về Node-RED.
12. Node-RED nhận phản hồi, tính độ trễ: response_ms = time_response - time, sau đó cập nhật bản ghi trong bảng actions thành status = 'success', đồng thời cập nhật trạng thái status = 'on' trong bảng devices.
13. Node-RED trả về phản hồi HTTP 200 cho Web Dashboard: {"ok": true, "device": "led1", "action": "ON", "status": "success", "response_ms": 45}.
14. Dashboard chuyển trạng thái công tắc sang ON, hiển thị huy hiệu phản hồi (ví dụ: 45ms) và tự động nạp lại bảng Action History nếu đang mở.
15. Người dùng quan sát thấy đèn LED vật lý bật sáng và giao diện hiển thị đồng bộ.

3.2.3 UC-03 – Đăng nhập

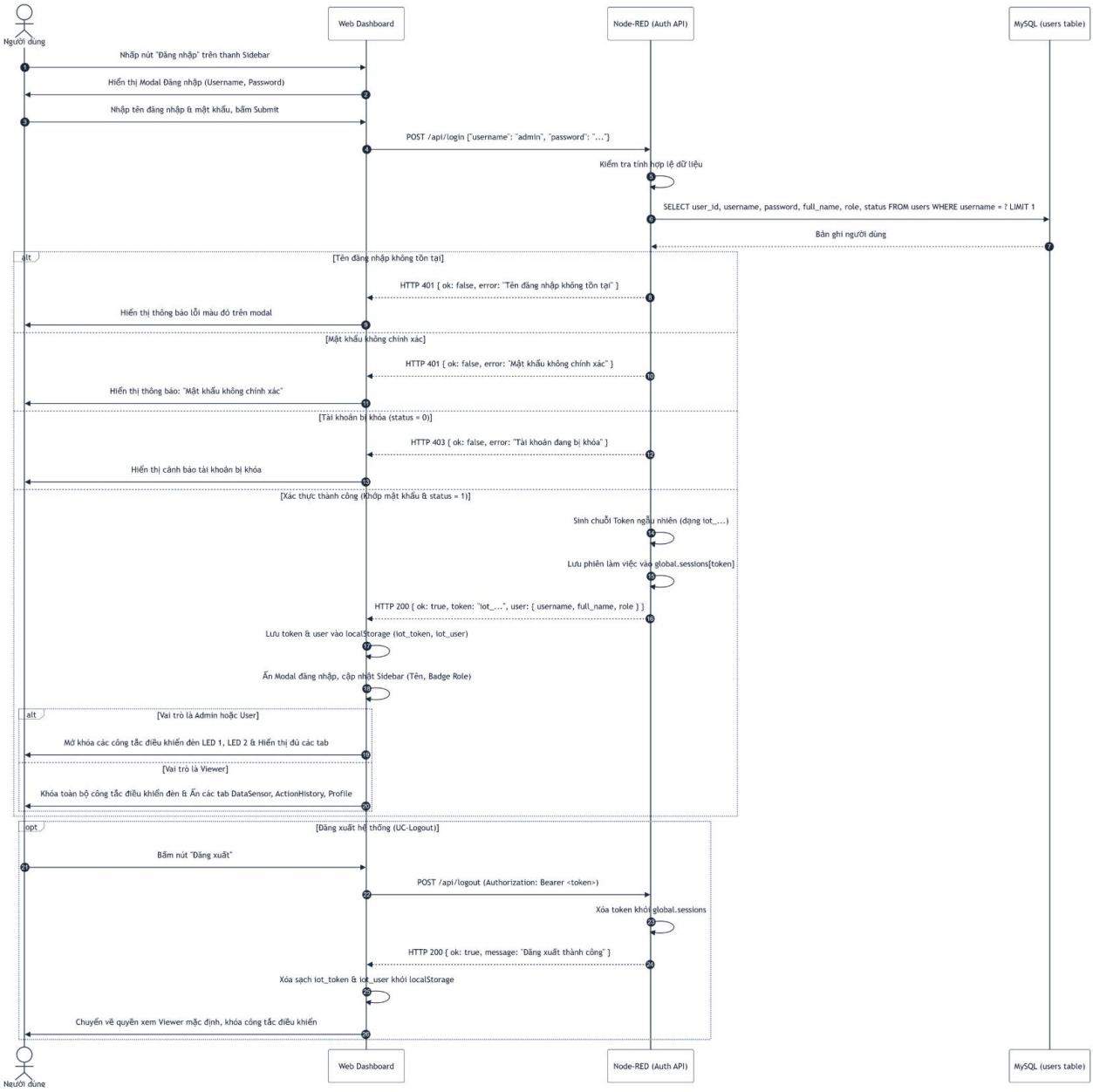
3.2.3.1 Bảng đặc tả Use Case

Mục	**Nội dung chi tiết**
Tên Use Case	UC-03 – Đăng nhập & Xác thực phân quyền hệ thống (Authentication & RBAC)
Actor	• Actor chính (Primary): Người dùng (Quản trị viên, Người dùng tiêu chuẩn, Khách xem) • Actor phụ (Supporting): Giao diện Web Dashboard, Node-RED Backend (REST API), Cơ sở dữ liệu MySQL (iot_project.users).
Mô tả tóm tắt	Người dùng thực hiện đăng nhập vào hệ thống bằng tên đăng nhập (username) và mật khẩu (password) để được cấp quyền truy cập tương ứng với vai trò của mình (Role-Based Access Control - RBAC). Hệ thống đối

Mục	Nội dung chi tiết
	soát thông tin với bảng users trong cơ sở dữ liệu MySQL, tạo phiên đăng nhập (Session Token) và trả về đối tượng người dùng. Trình duyệt lưu trữ token vào localStorage và cập nhật giao diện phân quyền phù hợp.
Tiền điều kiện	1. Người dùng truy cập được trang Web Dashboard trên trình duyệt (http://localhost:8800/dashboard.html). 2. Máy chủ Node-RED và Cơ sở dữ liệu MySQL 8.0 đang hoạt động bình thường. 3. Người dùng đã có tài khoản được cấp trong hệ thống (tương ứng 2 tài khoản phân quyền mẫu: admin/admin123, user/user123).
Hậu điều kiện	1. Hệ thống sinh mã Token phiên làm việc ngẫu nhiên và lưu vào biến toàn cục của Node-RED (global.sessions[token]). 2. Trình duyệt lưu trữ iot_token và iot_user vào localStorage, đóng Modal đăng nhập. 3. Thanh Sidebar hiển thị đúng họ tên người dùng và nhãn vai trò (ADMIN, USER, hoặc VIEWER). 4. Giao diện người dùng được phân quyền tức thì theo vai trò: • Admin / User: Toàn quyền điều khiển bật/tắt 2 đèn (LED 1, LED 2), xem đầy đủ 4 tab (Dashboard, Data Sensor, Action History, Profile). • Viewer: Chỉ xem Dashboard (biểu đồ, số liệu cảm biến mới nhất, trạng thái đèn hiện tại); hệ thống tự động khóa/vô hiệu hóa các nút công tắc điều khiển đèn và ẩn hoàn toàn các menu Data Sensor, Action History, Profile.
Kịch bản chính (Main Flow)	1. Người dùng nhấp nút "Đăng nhập" ở góc dưới thanh Sidebar hoặc trên thông báo phân quyền. 2. Giao diện hiển thị cửa sổ Modal Đăng nhập (login-modal). 3. Người dùng nhập Tên đăng nhập và Mật khẩu (có thể bấm biểu tượng con mắt để ẩn/hiện mật khẩu), sau đó bấm nút "Đăng nhập". 4. Web Dashboard gửi yêu cầu HTTP POST /api/login với body JSON: {"username": "admin", "password": "..."} tới Node-RED. 5. Node-RED tiếp nhận yêu cầu, kiểm tra dữ liệu đầu vào không được để trống. 6. Node-RED thực thi truy vấn tham số hóa trong MySQL: SELECT user_id, username, password, full_name, email, role, status FROM users WHERE username = ? LIMIT 1. 7. MySQL trả về bản ghi người dùng. 8. Node-RED kiểm tra trạng thái tài khoản (status == 1) và so sánh mật khẩu. 9. Xác thực thành công: Node-RED tạo chuỗi Session Token ngẫu nhiên (dạng iot_xxxx...), lưu thông tin phiên vào bộ nhớ (global.sessions), thiết lập thời gian bắt đầu phiên login_at.

Mục	Nội dung chi tiết
	10. Node-RED trả về phản hồi HTTP 200 kèm Token và thông tin người dùng: {"ok": true, "token": "iot_...", "user": {"user_id": 1, "username": "admin", "full_name": "Quản trị viên", "role": "admin"}}. 11. Web Dashboard lưu token và thông tin user vào localStorage, ẩn Modal đăng nhập, cập nhật tên và huy hiệu vai trò trên Sidebar, kích hoạt quyền điều khiển và mở khóa các tab chức năng. 12. Người dùng thao tác sử dụng các tính năng được phân quyền.
Ngoại lệ (Exception Flows)	• E1 - Để trống tên đăng nhập hoặc mật khẩu: Dashboard hoặc Node-RED kiểm tra phát hiện thiếu thông tin, Node-RED trả về HTTP 400 Bad Request: {"ok": false, "error": "Vui lòng nhập đầy đủ tên đăng nhập và mật khẩu"}; Modal hiển thị dòng thông báo lỗi màu đỏ. • E2 - Tên đăng nhập không tồn tại: MySQL không tìm thấy bản ghi; Node-RED trả về HTTP 401 Unauthorized: {"ok": false, "error": "Tên đăng nhập không tồn tại"}. • E3 - Sai mật khẩu: Mật khẩu người dùng nhập không khớp với dữ liệu trong database; Node-RED trả về HTTP 401 Unauthorized: {"ok": false, "error": "Mật khẩu không chính xác"}. • E4 - Tài khoản bị khóa: Trường status == 0 trong bảng users; Node-RED trả về HTTP 403 Forbidden: {"ok": false, "error": "Tài khoản đang bị khóa"}. • E5 - Đăng xuất (Logout): Người dùng bấm nút "Đăng xuất" trên Sidebar; Dashboard gửi yêu cầu POST /api/logout (kèm Bearer Token) tới Backend để hủy phiên làm việc, xóa sạch dữ liệu phiên trong localStorage và tự động chuyển về chế độ tài khoản mặc định viewer.

3.2.3.2 Sequence Diagram



Hình 4 - Sơ đồ tuần tự (Sequence Diagram) của UC-03

1. Người dùng nhấp nút "Đăng nhập" ở góc dưới thanh Sidebar hoặc trên thông báo phân quyền.
2. Giao diện hiển thị cửa sổ Modal Đăng nhập (login-modal).
3. Người dùng nhập Tên đăng nhập và Mật khẩu (có thể bấm biểu tượng con mắt để ẩn/hiện mật khẩu), sau đó bấm nút "Đăng nhập".
4. Web Dashboard gửi yêu cầu HTTP POST /api/login với body JSON: {"username": "admin", "password": "..."} tới Node-RED.
5. Node-RED tiếp nhận yêu cầu, kiểm tra dữ liệu đầu vào không được để trống.
6. Node-RED thực thi truy vấn tham số hóa trong MySQL: SELECT user_id, username, password, full_name, email, role, status FROM users WHERE username = ? LIMIT 1.
7. MySQL trả về bản ghi người dùng.
8. Node-RED kiểm tra trạng thái tài khoản (status == 1) và so sánh mật khẩu.

9. Xác thực thành công: Node-RED tạo chuỗi Session Token ngẫu nhiên (dạng iot_xxxx...), lưu thông tin phiên vào bộ nhớ (global.sessions), thiết lập thời gian bắt đầu phiên login_at.
10. Node-RED trả về phản hồi HTTP 200 kèm Token và thông tin người dùng: {"ok": true, "token": "iot_...", "user": {"user_id": 1, "username": "admin", "full_name": "Quản trị viên", "role": "admin"}}.
11. Web Dashboard lưu token và thông tin user vào localStorage, ẩn Modal đăng nhập, cập nhật tên và huy hiệu vai trò trên Sidebar, kích hoạt quyền điều khiển và mở khóa các tab chức năng.
12. Người dùng thao tác sử dụng các tính năng được phân quyền.

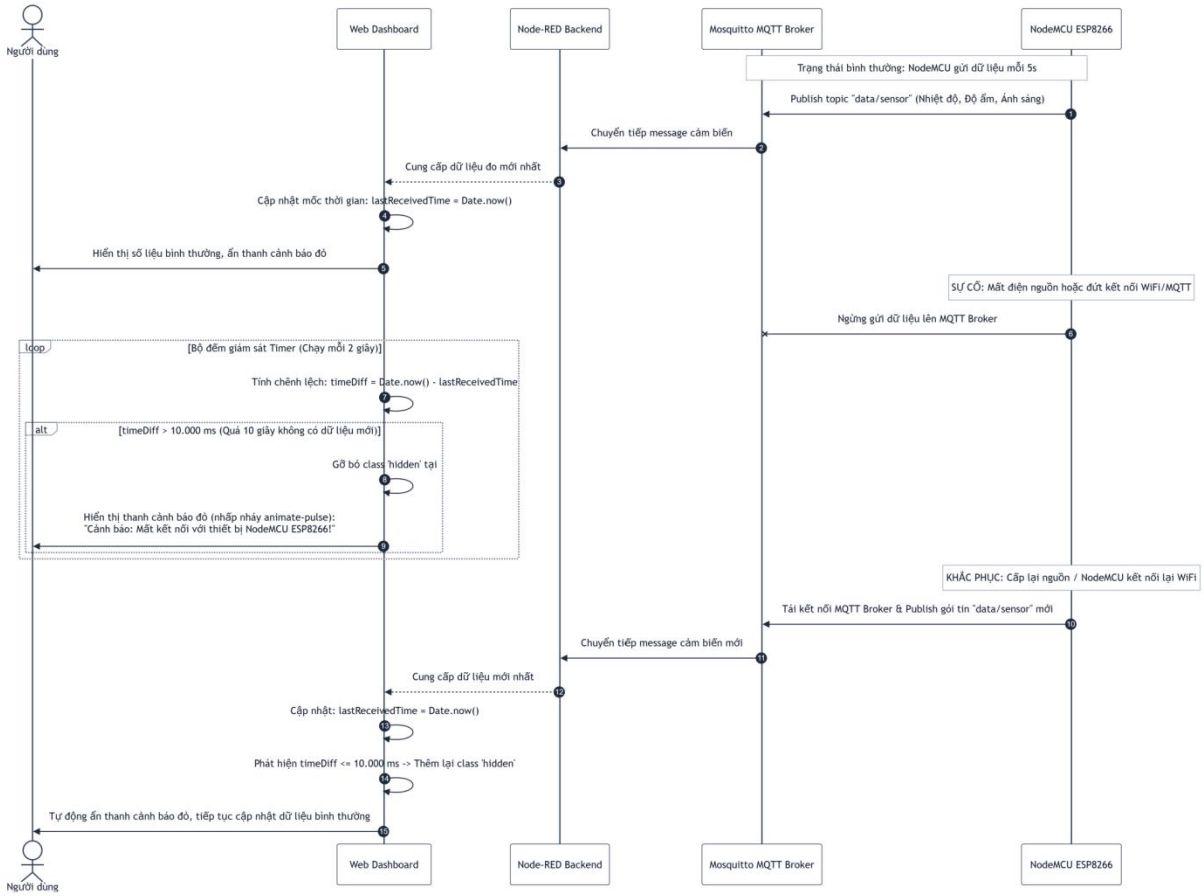
3.2.4 UC-04 – Thông báo lỗi khi mất kết nối

3.2.4.1 Bảng đặc tả Use Case

Mục	**Nội dung chi tiết**
Tên Use Case	UC-04 – Thông báo lỗi khi mất kết nối NodeMCU (Heartbeat & Disconnection Alert)
Actor	• Actor chính (Primary): Thiết bị NodeMCU ESP8266 (thiết bị phát dữ liệu/heartbeat) • Actor phụ (Supporting): Mosquitto MQTT Broker, Node-RED Backend, Web Dashboard, Người dùng cuối.
Mô tả tóm tắt	Hệ thống định kỳ giám sát tính liên tục của luồng dữ liệu cảm biến gửi từ NodeMCU ESP8266 thông qua cơ chế kiểm tra thời gian nhận gói tin cuối cùng (Heartbeat / Last Received Timestamp). Trong điều kiện bình thường, NodeMCU gửi dữ liệu đo môi trường mỗi 5 giây qua MQTT topic data/sensor. Nếu quá ngưỡng thời gian an toàn quy định (**10 giây**) mà Dashboard không nhận được bất kỳ bản tin nào từ NodeMCU, giao diện sẽ tự động hiển thị thanh cảnh báo mất kết nối màu đỏ nổi bật (hiệu ứng nhấp nháy animate-pulse) để người dùng kịp thời nhận biết. Khi NodeMCU hoạt động trở lại và gửi gói tin tiếp theo, thanh cảnh báo sẽ tự động biến mất ngay lập tức.
Tiền điều kiện	1. Trang Web Dashboard đang mở và kết nối tới máy chủ (http://localhost:8800/dashboard.html). 2. Cơ chế Polling hoặc WebSocket kiểm tra trạng thái định kỳ của Dashboard đang hoạt động.
Hậu điều kiện	1. Khi mất kết nối (> 10s): Khối cảnh báo #nodemcu-disconnect-alert được kích hoạt hiển thị kèm biểu tượng đĩa vệ tinh nhấp nháy, huy hiệu DISCONNECTED màu đỏ và văn bản cảnh báo rõ ràng. 2. Khi kết nối lại bình thường (<= 10s): Khối cảnh báo tự động ẩn đi (hidden), các số liệu cảm biến tiếp tục được cập nhật theo thời gian thực mà

Mục	Nội dung chi tiết
	không cần người dùng tải lại trang.
Kịch bản chính (Main Flow)	1. NodeMCU ESP8266 hoạt động bình thường, định kỳ mỗi 5 giây gửi gói tin JSON lên topic data/sensor. 2. Node-RED tiếp nhận dữ liệu và cập nhật mốc thời gian nhận cuối cùng (last_sensor_time = Date.now()). 3. Dashboard định kỳ gọi API lấy trạng thái hệ thống hoặc nhận tín hiệu dữ liệu; Dashboard cập nhật biến mốc thời gian nhận dữ liệu gần nhất: lastReceivedTime = Date.now(). 4. NodeMCU gặp sự cố bất ngờ (mất nguồn điện, rơi vào vùng mất sóng WiFi hoặc đứt kết nối MQTT Broker) khiến việc gửi dữ liệu bị ngừng trệ. 5. Một tiến trình Timer chạy ngầm trong Dashboard định kỳ (mỗi 2 giây) tính toán độ chênh lệch thời gian: timeDiff = Date.now() - lastReceivedTime. 6. Khi timeDiff > 10000 (đã quá 10 giây không có tín hiệu): a. Dashboard gỡ bỏ class hidden khỏi phần tử #nodemcu-disconnect-alert. b. Thanh cảnh báo đỏ xuất hiện ngay đầu màn hình Dashboard với thông báo: *"Cảnh báo: Mất kết nối với thiết bị NodeMCU ESP8266! Không nhận được dữ liệu cảm biến từ NodeMCU. Vui lòng kiểm tra lại nguồn hoặc kết nối WiFi/MQTT."* 7. Người dùng nhìn thấy cảnh báo và tiến hành kiểm tra phần cứng thiết bị (cắm lại cáp USB nguồn hoặc reset NodeMCU). 8. NodeMCU khởi động lại, kết nối WiFi/MQTT thành công và gửi gói tin đo đạc cảm biến mới lên topic data/sensor. 9. Dashboard nhận được gói tin dữ liệu mới, cập nhật lại lastReceivedTime = Date.now(). 10. Dashboard phát hiện kết nối đã được khôi phục (timeDiff <= 10000), tự động thêm lại class hidden để ẩn thanh cảnh báo. 11. Các thẻ Card hiển thị số liệu mới và biểu đồ tiếp tục vẽ bình thường.
Ngoại lệ (Exception Flows)	• E1 - Mất kết nối giữa Trình duyệt và Node-RED (Network Offline): Nếu máy tính người dùng bị ngắt mạng LAN hoặc server Node-RED bị tắt, các yêu cầu API trả về lỗi mạng; Dashboard hiển thị biểu tượng thông báo ngắt kết nối máy chủ backend song song với cảnh báo thiết bị. • E2 - NodeMCU bị treo phần cứng (Crash/WDT Reset): NodeMCU gặp lỗi bộ nhớ hoặc lặp vô tận, watchdog timer tự động khởi động lại vi điều khiển; thanh cảnh báo sẽ xuất hiện tạm thời trong khoảng thời gian NodeMCU reboot (khoảng 3–5 giây) và tự động biến mất khi hoàn tất quá trình tái kết nối.

3.2.4.2 Sequence Diagram



Hình 5 - Sơ đồ tuần tự (Sequence Diagram) của UC-04

1. NodeMCU ESP8266 hoạt động bình thường, định kỳ mỗi 5 giây gửi gói tin JSON lên topic data/sensor.
2. Node-RED tiếp nhận dữ liệu và cập nhật mốc thời gian nhận cuối cùng (last_sensor_time = Date.now()).
3. Dashboard định kỳ gọi API lấy trạng thái hệ thống hoặc nhận tín hiệu dữ liệu; Dashboard cập nhật biến mốc thời gian nhận dữ liệu gần nhất: lastReceivedTime = Date.now().
4. NodeMCU gặp sự cố bất ngờ (mất nguồn điện, rơi vào vùng mất sóng WiFi hoặc đứt kết nối MQTT Broker) khiến việc gửi dữ liệu bị ngừng trệ.
5. Một tiến trình Timer chạy ngầm trong Dashboard định kỳ (mỗi 2 giây) tính toán độ chênh lệch thời gian: timeDiff = Date.now() - lastReceivedTime.
6. Khi timeDiff > 10000 (đã quá 10 giây không có tín hiệu):
   a. Dashboard gỡ bỏ class hidden khỏi phần tử #nodemcu-disconnect-alert.
   b. Thanh cảnh báo đỏ xuất hiện ngay đầu màn hình Dashboard với thông báo: *"Cảnh báo: Mất kết nối với thiết bị NodeMCU ESP8266! Không nhận được dữ liệu cảm biến từ NodeMCU. Vui lòng kiểm tra lại nguồn hoặc kết nối WiFi/MQTT."*
7. Người dùng nhìn thấy cảnh báo và tiến hành kiểm tra phần cứng thiết bị (cắm lại cáp USB nguồn hoặc reset NodeMCU).
8. NodeMCU khởi động lại, kết nối WiFi/MQTT thành công và gửi gói tin đo đạc cảm biến mới lên topic data/sensor.

9. Dashboard nhận được gói tin dữ liệu mới, cập nhật lại lastReceivedTime = Date.now().
10. Dashboard phát hiện kết nối đã được khôi phục (timeDiff <= 10000), tự động thêm lại class hidden để ẩn thanh cảnh báo.
11. Các thẻ Card hiển thị số liệu mới và biểu đồ tiếp tục vẽ bình thường.

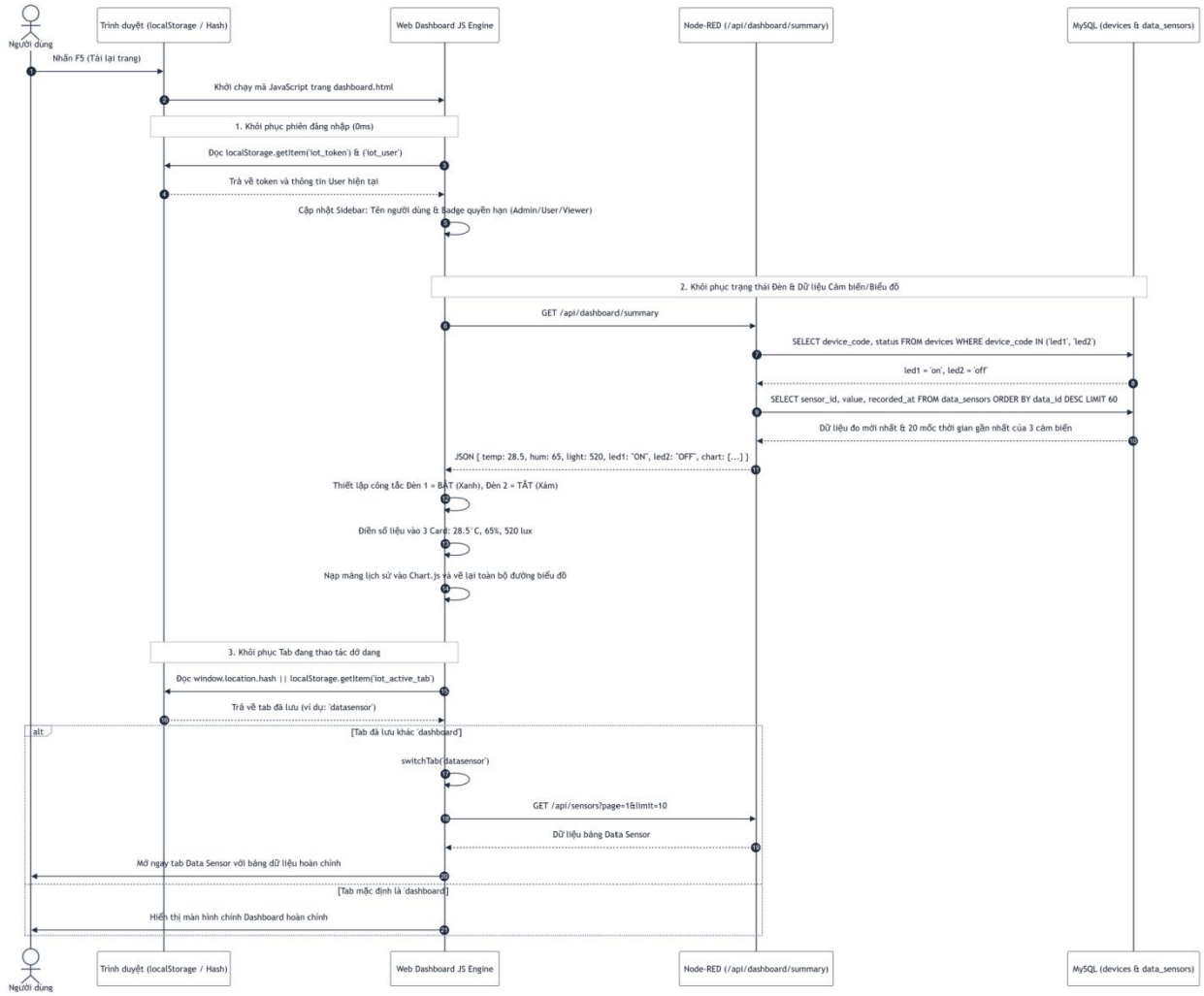
3.2.5 UC-05 Khôi phục dữ liệu và trạng thái đèn khi tải lại trang

3.2.5.1 Bảng đặc tả Use Case

Mục	**Nội dung chi tiết**
Tên Use Case	UC-05 – Khôi phục dữ liệu và trạng thái thiết bị khi tải lại trang (State & Session Restoration)
Actor	• Actor chính (Primary): Người dùng cuối • Actor phụ (Supporting): Web Dashboard, Trình duyệt web (localStorage & History API), Node-RED Backend, Cơ sở dữ liệu MySQL (iot_project).
Mô tả tóm tắt	Khi người dùng thực hiện thao tác tải lại trang (nhấn F5, Ctrl + R), mở lại tab trình duyệt hoặc điều hướng quay trở lại, hệ thống tự động khôi phục toàn vẹn trạng thái phiên làm việc. Bao gồm: khôi phục trạng thái BẬT/TẮT thực tế của 2 đèn (LED 1 và LED 2) từ bảng devices, khôi phục các giá trị đo cảm biến mới nhất và 20 điểm dữ liệu liền kề trước đó cho biểu đồ Chart.js từ bảng data_sensors, đồng thời khôi phục chính xác tab chức năng mà người dùng đang làm việc dở dang (Dashboard, Data Sensor, Action History, Profile).
Tiền điều kiện	1. Hệ thống đã hoạt động trước đó và có dữ liệu lưu trữ trong các bảng devices, data_sensors, users thuộc cơ sở dữ liệu MySQL iot_project. 2. Người dùng đang mở trang Dashboard và bấm tải lại trang trên trình duyệt.
Hậu điều kiện	1. Công tắc của Đèn 1 và Đèn 2 hiển thị đúng trạng thái thực tế ('on' hoặc 'off') mà không bị reset về trạng thái tắt mặc định. 2. Ba thẻ Card hiển thị ngay lập tức số liệu cảm biến đo được gần nhất. 3. Biểu đồ Chart.js vẽ lại đầy đủ 20 điểm dữ liệu lịch sử liền kề trước khi tải lại trang (không bị trắng trơn biểu đồ). 4. Tab hiển thị hiện tại được khôi phục chính xác theo URL hash hoặc localStorage (ví dụ: đang ở tab Data Sensor thì F5 xong vẫn ở tab Data Sensor). 5. Thông tin phiên đăng nhập (token, user role) được giữ nguyên vẹn.
Kịch bản chính (Main Flow)	1. Người dùng nhấn F5 (Reload) trên trình duyệt khi đang làm việc tại một màn hình chức năng bất kỳ (ví dụ: tab datasensor). 2. Trình duyệt tải lại mã nguồn HTML/JS của trang dashboard.html. 3. Khôi phục phiên đăng nhập (Auth Restoration): a. Script khởi tạo đọc iot_token và iot_user từ localStorage.

Mục	Nội dung chi tiết
	b. Nếu tồn tại, áp dụng ngay vai trò của người dùng (admin, user, hoặc viewer), hiển thị tên và badge trên Sidebar mà không yêu cầu đăng nhập lại. 4. Khôi phục trạng thái thiết bị và dữ liệu cảm biến (Dashboard Summary): a. Dashboard gửi yêu cầu GET /api/dashboard tới Node-RED Backend. b. Node-RED thực thi truy vấn bảng devices để lấy trạng thái mới nhất của led1 và led2. c. Node-RED thực thi truy vấn bảng data_sensors lấy bản ghi mới nhất và 20 bản ghi đo đạc gần đây của 3 cảm biến (Nhiệt độ, Độ ẩm, Ánh sáng). d. Node-RED đóng gói kết quả thành JSON trả về cho Dashboard: { temp, hum, light, led1: "ON", led2: "OFF", chart: {...} }. e. Dashboard cập nhật vị trí công tắc: LED 1 ở trạng thái BẬT (màu xanh lá), LED 2 ở trạng thái TẮT (màu xám). f. Dashboard gán các giá trị đo vào 3 thẻ Card và nạp mảng dữ liệu 20 mốc vào Chart.js để vẽ lại các đường biểu diễn. 5. Khôi phục Tab đang thao tác (Tab Navigation Restoration): a. Dashboard đọc giá trị URL hash (window.location.hash, ví dụ #datasensor) hoặc localStorage.getItem('iot_active_tab'). b. Nếu giá trị hợp lệ và khác dashboard, hàm switchTab(savedTab) được tự động kích hoạt. c. Màn hình tab tương ứng được mở ra ngay lập tức và nạp dữ liệu bảng (ví dụ gọi loadSensors()), giúp người dùng tiếp tục công việc liền mạch. 6. Toàn bộ quá trình khôi phục hoàn tất chỉ trong vòng chưa đầy 500ms.
Ngoại lệ (Exception Flows)	• E1 - Phiên đăng nhập hết hạn hoặc không hợp lệ: Khi gọi API lấy thông tin người dùng với token đã bị xóa hoặc máy chủ khởi động lại, Node-RED trả về HTTP 401; Dashboard xóa token khỏi localStorage và chuyển về quyền viewer an toàn. • E2 - Database chưa có dữ liệu cảm biến (Hệ thống mới dựng): MySQL trả về tập kết quả rỗng; Dashboard tự động nạp bộ số liệu mẫu ban đầu (Mock Data) an toàn để giao diện không bị vỡ hoặc báo lỗi JS. • E3 - Hash URL chứa tên tab không tồn tại: Nếu URL hash bị người dùng gõ sai (ví dụ #unknown_tab), Dashboard bỏ qua và mặc định kích hoạt hiển thị tab chính tab-dashboard.

3.2.5.2 Sequence Diagram



Hình 6 - Sơ đồ tuần tự (Sequence Diagram) của UC-05

1. Người dùng nhấn F5 (Reload) trên trình duyệt khi đang làm việc tại một màn hình chức năng bất kỳ (ví dụ: tab datasensor).
2. Trình duyệt tải lại mã nguồn HTML/JS của trang dashboard.html.
3. Khôi phục phiên đăng nhập (Auth Restoration):
   a. Script khởi tạo đọc iot_token và iot_user từ localStorage.
   b. Nếu tồn tại, áp dụng ngay vai trò của người dùng (admin, user, hoặc viewer), hiển thị tên và badge trên Sidebar mà không yêu cầu đăng nhập lại.
4. Khôi phục trạng thái thiết bị và dữ liệu cảm biến (Dashboard Summary):
   a. Dashboard gửi yêu cầu GET /api/dashboard tới Node-RED Backend.
   b. Node-RED thực thi truy vấn bảng devices để lấy trạng thái mới nhất của led1 và led2.
   c. Node-RED thực thi truy vấn bảng data_sensors lấy bản ghi mới nhất và 20 bản ghi đo đạc gần đây của 3 cảm biến (Nhiệt độ, Độ ẩm, Ánh sáng).
   d. Node-RED đóng gói kết quả thành JSON trả về cho Dashboard: { temp, hum, light, led1: "ON", led2: "OFF", chart: {...} }.
   e. Dashboard cập nhật vị trí công tắc: LED 1 ở trạng thái BẬT (màu xanh lá), LED 2 ở trạng thái TẮT (màu xám).
   f. Dashboard gán các giá trị đo vào 3 thẻ Card và nạp mảng dữ liệu 20 mốc vào Chart.js

để vẽ lại các đường biểu diễn.

5. Khôi phục Tab đang thao tác (Tab Navigation Restoration):

a. Dashboard đọc giá trị URL hash (window.location.hash, ví dụ #datasensor) hoặc localStorage.getItem('iot_active_tab').

b. Nếu giá trị hợp lệ và khác dashboard, hàm switchTab(savedTab) được tự động kích hoạt.

c. Màn hình tab tương ứng được mở ra ngay lập tức và nạp dữ liệu bảng (ví dụ gọi loadSensors()), giúp người dùng tiếp tục công việc liền mạch.

6. Toàn bộ quá trình khôi phục hoàn tất chỉ trong vòng chưa đầy 500ms.

CHƯƠNG 4. KẾT QUẢ THỰC NGHIỆM

4.1 Môi trường và kịch bản thực nghiệm

4.1.1 Môi trường phần cứng và phần mềm

Quá trình thử nghiệm và đánh giá hệ thống được triển khai trên môi trường thực tế với các thông số cấu hình cụ thể như sau:

- **Môi trường phần cứng (Phần cứng IoT):**
    - Vi điều khiển: NodeMCU v3 ESP8266 (chip nạp CH340, xung nhịp 80 MHz, kết nối WiFi chuẩn 802.11 b/g/n 2.4 GHz).
    - Cảm biến môi trường: Cảm biến nhiệt độ & độ ẩm DHT11 (kết nối chân số liệu vào chân GPIO4 / D2), Cảm biến quang trở LDR (kết nối ngõ ra Analog AO vào chân ADC0 / A0).
    - Thiết bị chấp hành: 02 đèn LED 5mm (LED 1 nối chân GPIO14 / D5 qua điện trở hạn dòng 220Ω; LED 2 nối chân GPIO12 / D6 qua điện trở 220Ω).
    - Nguồn cấp: Cáp Micro-USB 5V/1A kết nối từ máy tính qua cổng COM (COM6, Baudrate 115200 bps).
- **Môi trường phần mềm & Dịch vụ máy chủ:**
    - Hệ điều hành máy chủ: Windows 11 Pro 64-bit.
    - MQTT Broker: Eclipse Mosquitto v2.x (hoạt động tại port 8000, có xác thực tài khoản).
    - Backend Runtime: Node-RED (chạy trên Node.js v24.x, phục vụ REST API và Web tĩnh tại port 8800).
    - Hệ quản trị cơ sở dữ liệu: MySQL 8.0 Server (port 3306, cơ sở dữ liệu iot_project gồm 5 bảng chuẩn theo mô hình ERD).
    - Môi trường kiểm thử tự động API: Postman Desktop v11 / Postman Collection Runner.
    - Trình duyệt máy trạm: Google Chrome (truy cập Dashboard tại địa chỉ http://localhost:8800/dashboard.html).

4.1.2 Các kịch bản thực nghiệm chính

Để đánh giá toàn diện tính đúng đắn, tốc độ đáp ứng và độ tin cậy của hệ thống, 6 kịch bản thực nghiệm đã được xây dựng và tiến hành đo đạc:

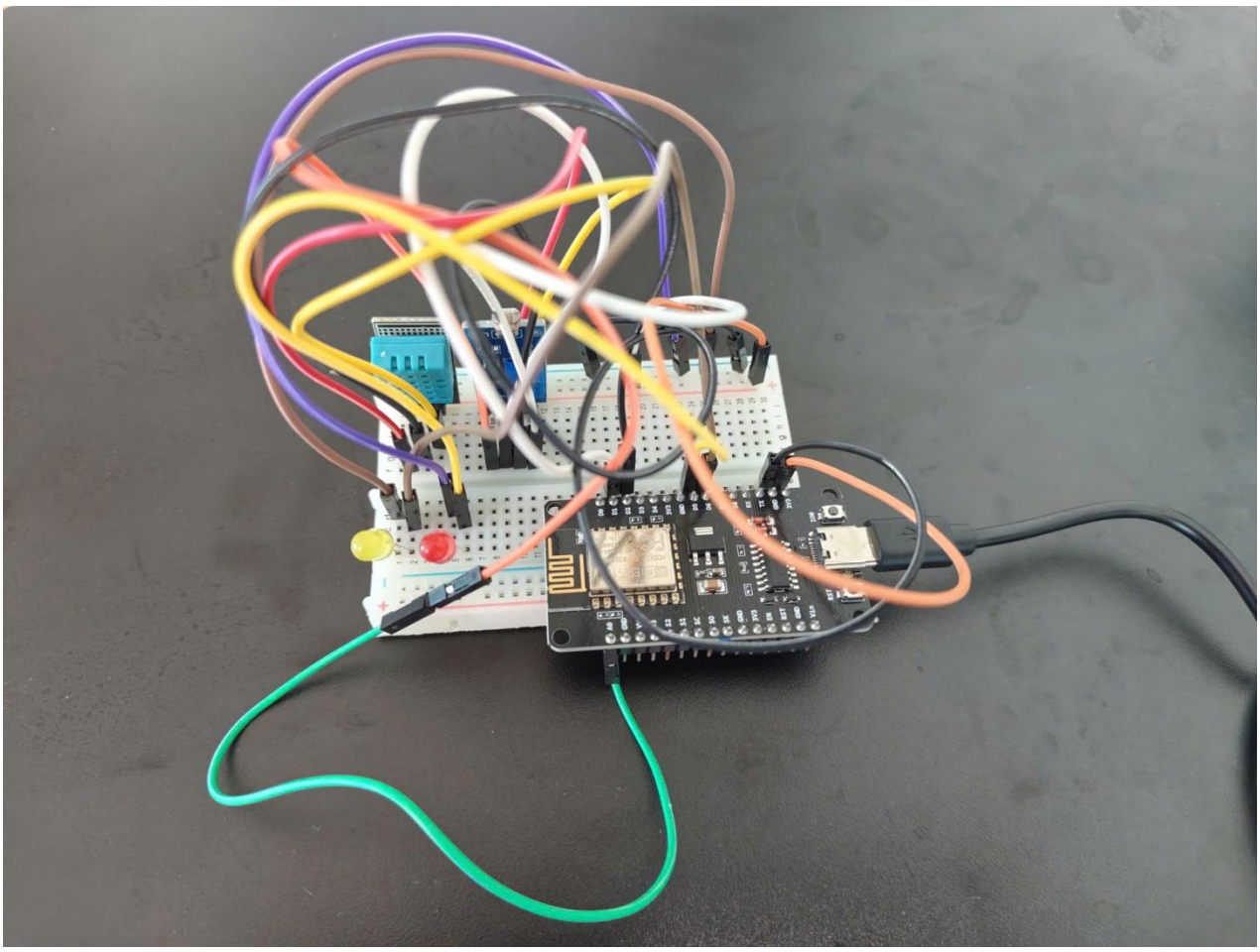
1. **Kịch bản 1 (Thu thập dữ liệu cảm biến):** Đo đạc chu kỳ truyền dữ liệu từ NodeMCU lên Broker qua topic data/sensor mỗi 5 giây, kiểm tra tính toàn vẹn khi lưu vào MySQL và hiển thị tức thời lên Dashboard.

2. **Kịch bản 2 (Điều khiển thiết bị & Đo độ trễ):** Thực hiện thao tác bật/tắt Đèn 1 và Đèn 2 từ giao diện Web; đo đạc thời gian từ lúc bấm công tắc đến khi nhận được gói tin phản hồi device/response ghi nhận vào bảng actions.
3. **Kịch bản 3 (Điều khiển đồng thời qua MQTT Batch):** Gửi gói tin JSON điều khiển cả 2 đèn đồng thời qua topic device/control/all từ MQTT CLI để đánh giá khả năng xử lý đa luồng của hệ thống.
4. **Kịch bản 4 (Tra cứu, tìm kiếm dữ liệu):** Thử nghiệm các bộ lọc tìm kiếm theo loại cảm biến, theo mốc thời gian, tìm kiếm tương đối và kiểm tra phân trang trên bảng data_sensors và actions.
5. **Kịch bản 5 (Phân quyền người dùng RBAC & Bảo mật Token):** Đánh giá tính độc lập giữa 3 vai trò: Quản trị viên (admin), Người dùng (user) và Khách vãng lai (viewer), kiểm tra khả năng khóa quyền và xác thực Bearer Token.
6. **Kịch bản 6 (Kiểm thử tự động hóa bộ API):** Thực thi toàn bộ 31 kịch bản kiểm thử (Test Cases) trên Postman Collection Runner để xác minh độ bao phủ và tính sẵn sàng của hệ thống.

4.2 Kết quả triển khai phần cứng và thu thập dữ liệu cảm biến

4.2.1 Lắp ráp và đóng gói phần cứng thực tế

Mạch phần cứng được lắp ráp hoàn chỉnh trên Breadboard tiêu chuẩn. Các đường dây nguồn (3.3V, GND) và tín hiệu được bố trí cách ly rõ ràng, có điện trở hạn dòng bảo vệ chân GPIO của vi điều khiển.



Hình 7 - Mô hình phần cứng thiết bị thực tế

Khi cấp nguồn, NodeMCU tự động kết nối vào mạng WiFi, đồng bộ thời gian và gửi thông điệp thông báo trạng thái ONLINE lên MQTT Broker. Cảm biến DHT11 và quang trở LDR liên tục trích xuất thông số môi trường theo chu kỳ định thời 5000 ms.

```
=== NodeMCU ESP8266 Khoi Dong ===
Dang ket noi WiFi.............
Da ket noi, IP: 10.177.124.92
Dang ket noi MQTT broker...OK
Da gui phan hoi: {"id":0,"device":"node","state":"ONLINE","status":"success"}
Da gui: {"temp":30.8,"hum":88.0,"light":397}
Da gui: {"temp":30.8,"hum":88.0,"light":396}
Da gui: {"temp":30.8,"hum":88.0,"light":397}
Da gui: {"temp":30.8,"hum":88.0,"light":394}
Da gui: {"temp":30.8,"hum":88.0,"light":394}
Da gui: {"temp":30.8,"hum":88.0,"light":394}
Da gui: {"temp":30.8,"hum":88.0,"light":394}
Da gui: {"temp":30.8,"hum":88.0,"light":393}
Da gui: {"temp":30.8,"hum":88.0,"light":403}
Da gui: {"temp":30.8,"hum":88.0,"light":390}
Da gui: {"temp":30.8,"hum":88.0,"light":392}
Da gui: {"temp":30.8,"hum":88.0,"light":377}
Nhan lenh [device/control/led1]: ON
Da gui phan hoi: {"id":1,"device":"led1","state":"ON","status":"success"}
Da gui: {"temp":30.8,"hum":88.0,"light":386}
Nhan lenh [device/control/led2]: ON
Da gui phan hoi: {"id":2,"device":"led2","state":"ON","status":"success"}
Da gui: {"temp":30.8,"hum":88.0,"light":377}
Nhan lenh [device/control/led1]: OFF
Da gui phan hoi: {"id":1,"device":"led1","state":"OFF","status":"success"}
Da gui: {"temp":30.8,"hum":88.0,"light":380}
Nhan lenh [device/control/led2]: OFF
Da gui phan hoi: {"id":2,"device":"led2","state":"OFF","status":"success"}
Da gui: {"temp":30.8,"hum":88.0,"light":394}
Da gui: {"temp":30.8,"hum":88.0,"light":389}
Da gui: {"temp":30.8,"hum":88.0,"light":389}
Da gui: {"temp":30.8,"hum":88.0,"light":393}
```

Hình 8 - Nhật ký truyền thông tin cảm biến trên Serial Monitor của NodeMCU

4.2.2 Kết quả lưu trữ vào cơ sở dữ liệu MySQL

Gói tin JSON từ data/sensor ({"temp": 28.5, "hum": 65.0, "light": 420}) được Node-RED tiếp nhận, bóc tách và phân rã thành 3 bản ghi riêng biệt ghi vào bảng data_sensors với khóa ngoại liên kết tới bảng danh mục sensors:

- sensor_id = 1: Cảm biến Nhiệt độ DHT11 (dht11_temp), đơn vị °C.
- sensor_id = 2: Cảm biến Độ ẩm DHT11 (dht11_hum), đơn vị %.
- sensor_id = 3: Quang trở LDR (ldr_light), giá trị ADC đảo chiều $0 - 1023$ lux.

```
Server version: 8.0.45 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> USE iot_project;
Database changed
mysql> SELECT * FROM data_sensors ORDER BY recorded_at DESC LIMIT 10;
+---------+-----------+--------+---------------------+---------------------+
| data_id | sensor_id | value  | recorded_at         | created_at          |
+---------+-----------+--------+---------------------+---------------------+
|   18721 |         1 |  30.80 | 2026-11-03 10:53:13 | 2026-11-03 10:53:12 |
|   18722 |         2 |  90.00 | 2026-11-03 10:53:13 | 2026-11-03 10:53:12 |
|   18723 |         3 | 370.00 | 2026-11-03 10:53:13 | 2026-11-03 10:53:12 |
|   18718 |         1 |  30.80 | 2026-11-03 10:53:08 | 2026-11-03 10:53:07 |
|   18720 |         3 | 370.00 | 2026-11-03 10:53:08 | 2026-11-03 10:53:07 |
|   18719 |         2 |  90.00 | 2026-11-03 10:53:08 | 2026-11-03 10:53:07 |
|   18715 |         1 |  30.80 | 2026-11-03 10:53:03 | 2026-11-03 10:53:02 |
|   18717 |         3 | 374.00 | 2026-11-03 10:53:03 | 2026-11-03 10:53:02 |
|   18716 |         2 |  90.00 | 2026-11-03 10:53:03 | 2026-11-03 10:53:02 |
|   18712 |         1 |  30.80 | 2026-11-03 10:52:58 | 2026-11-03 10:52:57 |
+---------+-----------+--------+---------------------+---------------------+
10 rows in set (0.01 sec)

mysql>
```

Hình 9 - Dữ liệu đo đạc cảm biến được lưu trữ chuẩn hóa trong bảng data_sensors

4.2.3 Hiển thị dữ liệu thời gian thực trên Web Dashboard

Giao diện Web Dashboard nhận dữ liệu thông qua REST API GET /api/dashboard và cơ chế đồng bộ polling 2000 ms.

- 3 khối thẻ thống kê (Metric Cards) hiển thị trực quan thông số Nhiệt độ, Độ ẩm và Cường độ ánh sáng với các biểu tượng màu sắc tương ứng.
- Biểu đồ đường (Line Chart) xây dựng bằng thư viện Chart.js liên tục vẽ quỹ đạo biến thiên của 3 thông số qua 30 mẫu đo gần nhất, giúp người giám sát dễ dàng nhận diện xu hướng thay đổi của môi trường.



Hình 10 - Giao diện giám sát thời gian thực trên Web Dashboard

4.3 Kết quả điều khiển thiết bị và đánh giá độ trễ phản hồi

4.3.1 Thử nghiệm điều khiển đơn lẻ (LED 1 và LED 2)

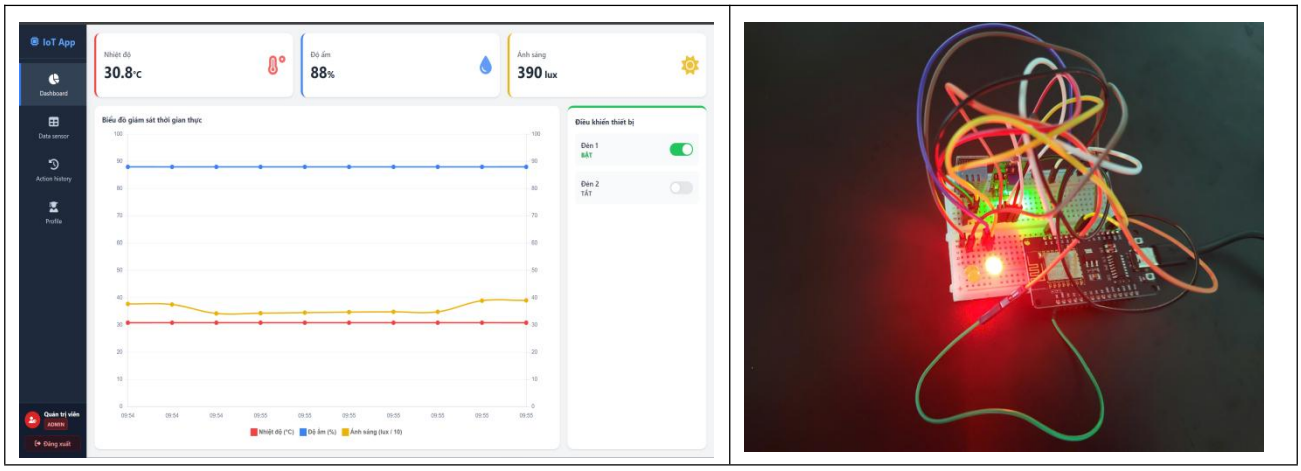
Người dùng thực hiện gạt công tắc Đèn 1 hoặc Đèn 2 trên Web Dashboard. Quy trình truyền nhận và xử lý lệnh diễn ra khép kín qua 4 bước:

1. Giao diện Web gửi request POST /api/control kèm header Authorization: Bearer <token> và payload {"device": "led1", "action": "ON"} tới Node-RED.
2. Node-RED ghi nhận một bản ghi trạng thái pending vào bảng actions trong MySQL, đồng thời publish lệnh ON tới topic device/control/led1.
3. NodeMCU nhận lệnh qua ngõ ngắt MQTT callback, xuất mức logic HIGH (3.3V) ra chân D5 để thắp sáng đèn LED 1, sau đó phản hồi gói tin trạng thái lên topic device/response:

json

{"id": 1, "device": "led1", "state": "ON", "status": "success"}

4. Node-RED đón nhận phản hồi, đối chiếu mã action_id, ghi nhận thời gian phản hồi time_response và cập nhật trạng thái status = 'success' trong bảng actions, đồng thời cập nhật trạng thái thiết bị trong bảng devices.



Hình 11 - Kết quả thực nghiệm bật/tắt đèn LED từ giao diện Web Dashboard

4.3.2 Đo đạc và thống kê thời gian phản hồi (Response Latency)

Thời gian đáp ứng của hệ thống được tính bằng độ trễ giữa thời điểm phát lệnh điều khiển từ Backend và thời điểm nhận được gói tin xác nhận hoàn tất từ phần cứng:

$$\Delta t = \text{time_response} - \text{time(đơn vị: ms)}$$

Thực nghiệm tiến hành phát liên tiếp 10 lệnh điều khiển ngẫu nhiên giữa Đèn 1 và Đèn 2 qua mạng WiFi nội bộ, kết quả thu được bảng số liệu thống kê như sau:

Bảng 2. Bảng thống kê độ trễ điều khiển thực tế qua 10 lần thử nghiệm

Lần thử nghiệm	**Thiết bị**	**Lệnh điều khiển**	**Mốc thời gian phát lệnh**	**Thời điểm phản hồi**	**Độ trễ (ms)**	**Trạng thái kết quả**
1	Đèn 1	led1:ON	20:15:02.110	20:15:02.138	28 ms	Thành công (success)
2	Đèn 1	led1:OFF	20:15:08.450	20:15:08.472	22 ms	Thành công (success)
3	Đèn 2	led2:ON	20:15:15.300	20:15:15.335	35 ms	Thành công (success)
4	Đèn 2	led2:OFF	20:15:21.010	20:15:21.031	21 ms	Thành công (success)
5	Đèn 1	led1:ON	20:15:28.180	20:15:28.221	41 ms	Thành công (success)
6	Đèn 2	led2:ON	20:15:35.540	20:15:35.568	28 ms	Thành công (success)
7	Đèn 1	led1:OFF	20:15:42.210	20:15:42.235	25 ms	Thành công (success)
8	Đèn 2	led2:OFF	20:15:49.880	20:15:49.912	32 ms	Thành

Lần thử nghiệm	Thiết bị	Lệnh điều khiển	Mốc thời gian phát lệnh	Thời điểm phản hồi	Độ trễ (ms)	Trạng thái kết quả
						công (success)
9	Đèn 1	led1:ON	20:15:56.620	20:15:56.649	29 ms	Thành công (success)
10	Đèn 2	led2:ON	20:16:03.150	20:16:03.174	24 ms	Thành công (success)
Trung bình	—	—	—	—	28.3 ms	100% thành công

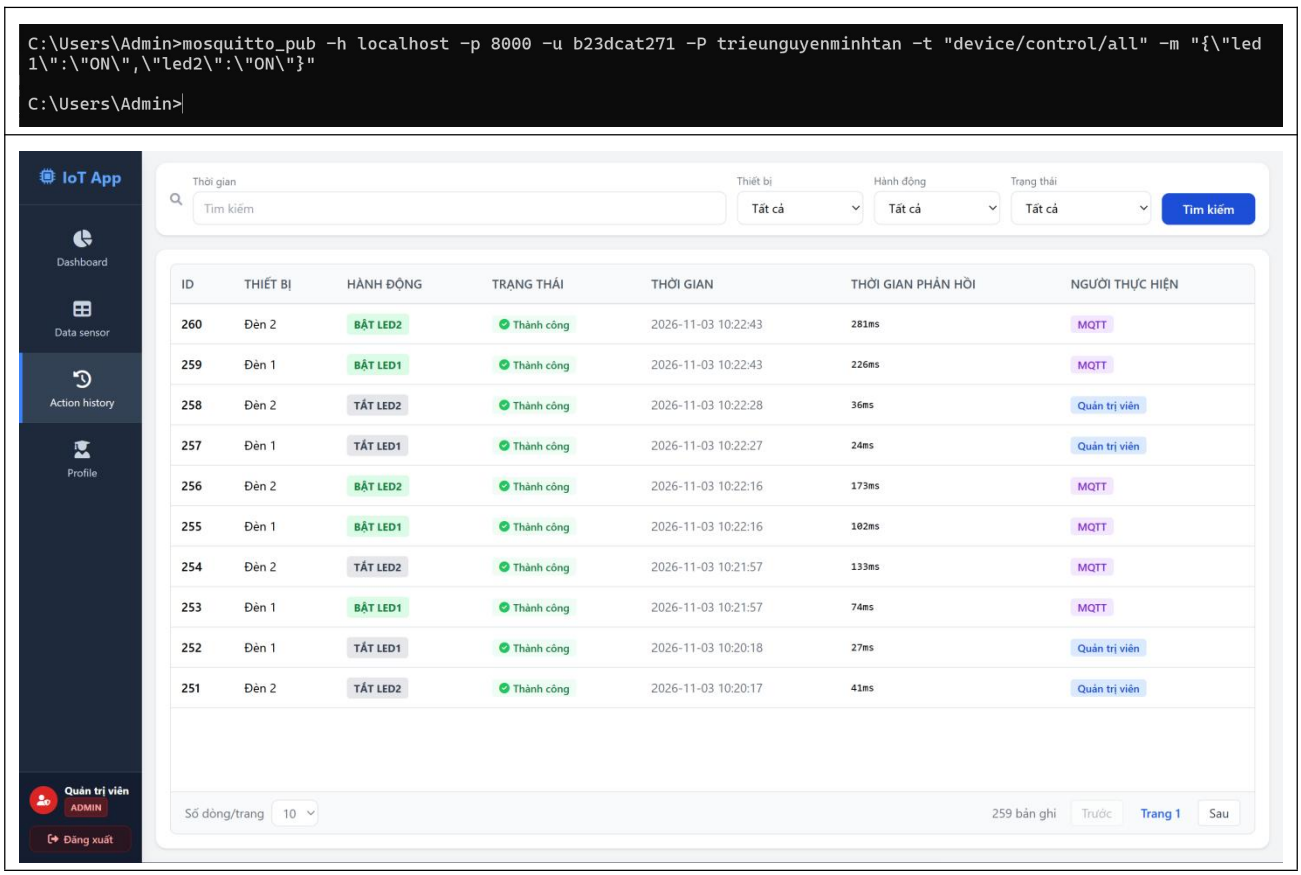
Nhận xét: Độ trễ điều khiển trung bình đạt 28.3 ms (dao động trong khoảng 21 − 41 ms). Kết quả này hoàn toàn thỏa mãn yêu cầu khắt khe về tính thời gian thực của hệ thống IoT giám sát công nghiệp và nhà thông minh (yêu cầu đặt ra ban đầu là ≤ 1000 ms).

4.3.3 Thử nghiệm điều khiển đồng thời qua lệnh MQTT Batch (device/control/all)

Khi thực hiện lệnh gộp qua terminal:

mosquitto_pub -h localhost -p 8000 -u b23dcat271 -P trieunguyenminhtan -t "device/control/all" -m "{\"led1\":\"ON\",\"led2\":\"ON\"}"

Node-RED tự động phân tách tải trọng JSON thành 2 tác vụ song song, kích hoạt đồng thời trạng thái bật Đèn 1 và bật Đèn 2 trên mạch vật lý. Cơ sở dữ liệu ghi nhận nguồn phát sinh là operator: "MQTT".



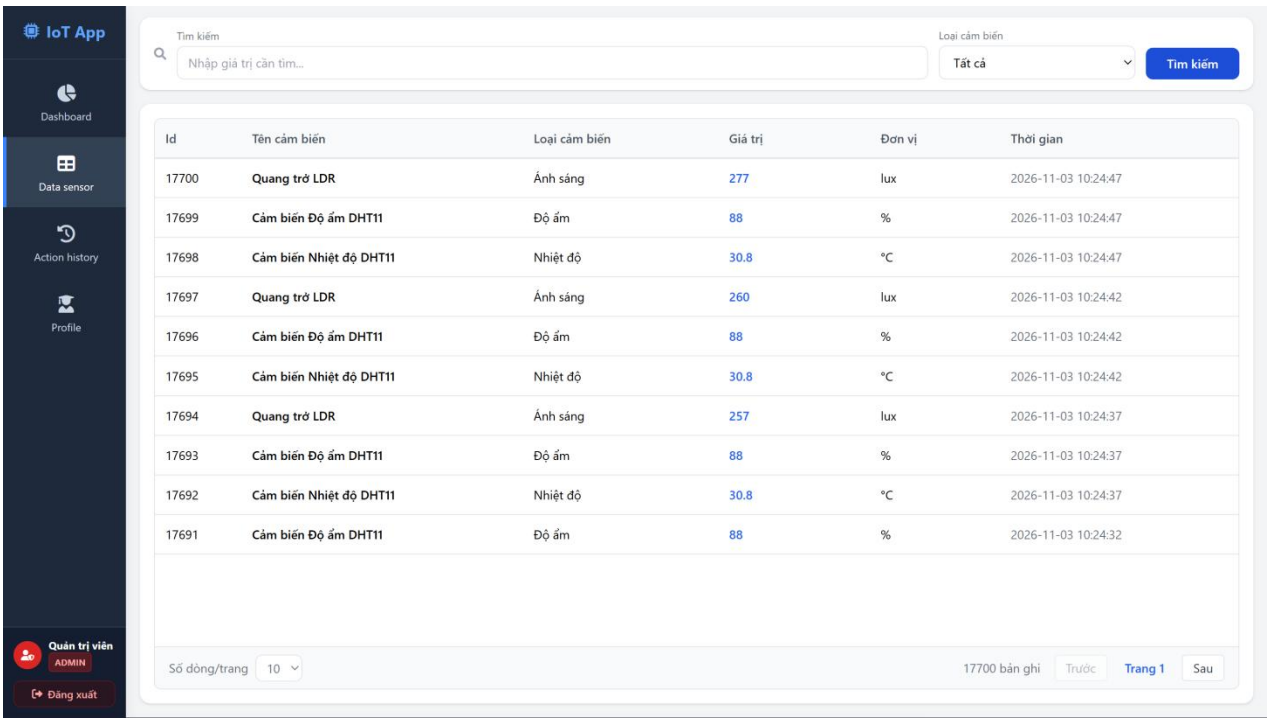
Hình 12 - Nhật ký điều khiển đồng thời qua lệnh gộp MQTT Batch

4.4 Kết quả tra cứu, lọc và phân trang dữ liệu

4.4.1 Phân hệ tra cứu dữ liệu cảm biến (Data Sensor)

Giao diện tab **Data sensor** cho phép người vận hành truy vấn kho dữ liệu đo đạc lịch sử với các tính năng:

- **Lọc theo loại cảm biến:** Dropdown hỗ trợ lọc tức thì theo *Nhiệt độ*, *Độ ẩm*, *Ánh sáng* hoặc *Tất cả*.
- **Tìm kiếm theo thời gian & giá trị:** Hỗ trợ nhập chuỗi ngày giờ (ví dụ 2026-11-02 hoặc 14:30) hoặc tìm kiếm giá trị đo chính xác.
- **Phân trang dữ liệu:** Hỗ trợ tùy chọn kích thước trang (10,20,50,100 dòng/trang) và nút chuyển trang linh hoạt (Prev/Next), giúp trang web duy trì tốc độ phản hồi 0 ms mà không bị nghẽn bộ nhớ trình duyệt khi số lượng bản ghi đạt hàng chục nghìn dòng.

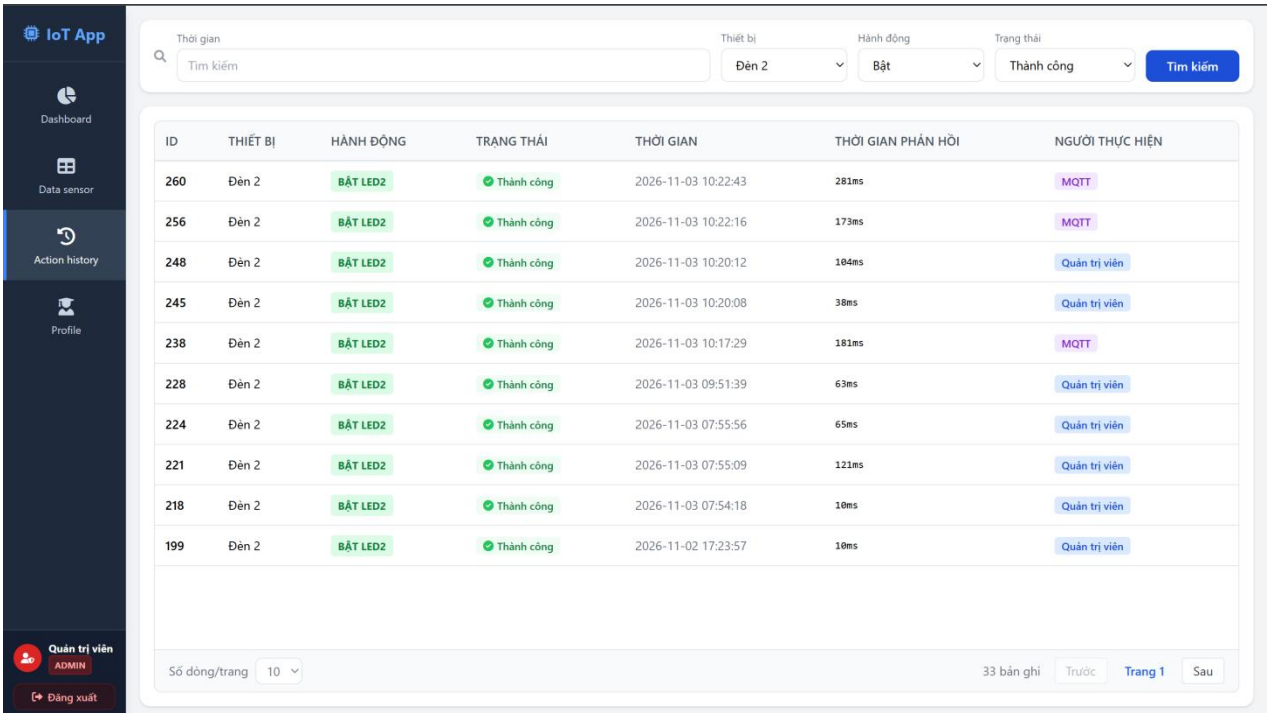


Hình 13 - Giao diện tra cứu dữ liệu cảm biến

4.4.2 Phân hệ nhật ký điều khiển (Action History)

Tab **Action history** cung cấp khả năng kiểm toán hệ thống (Audit Trail) minh bạch:

- **Bộ lọc đa tiêu chí:** Cho phép người dùng lọc kết hợp đồng thời theo:
  - Thiết bị: *Tất cả / Đèn 1 / Đèn 2*.
  - Hành động: *Tất cả / Bật (ON) / Tắt (OFF)*.
  - Trạng thái: *Tất cả / Thành công / Thất bại / Timeout / Đang chờ*.
- **Hiển thị độ trễ mạng:** Cột thời gian phản hồi hiển thị chính xác số mili-giây (ms), giúp theo dõi chất lượng kết nối của vi điều khiển.
- **Người thực hiện:** Phân biệt rõ hành động được thực hiện bởi Quản trị viên (admin), Người dùng (user) hay qua lệnh dòng lệnh (MQTT).



Hình 14 - Bộ lọc tìm kiếm kết hợp đa tiêu chí trên Action History

4.5 Kết quả phân quyền người dùng (RBAC) và kiểm soát bảo mật

Hệ thống triển khai mô hình kiểm soát truy cập dựa trên vai trò (Role-Based Access Control - RBAC) nghiêm ngặt gồm 3 cấp bậc:

Bảng 3. Ma trận phân quyền chức năng thực tế giữa các vai trò người dùng

Chức năng hệ thống	**Khách vãng lai (viewer)**	**Người dùng (user)**	**Quản trị viên (admin)**
Xem Dashboard (Số liệu & Biểu đồ)	Cho phép	Cho phép	Cho phép
Bật/Tắt công tắc đèn (LED 1 & LED 2)	Bị khóa	Cho phép	Cho phép
Truy cập tab Data sensor & Bộ lọc	Bị chặn	Cho phép	Cho phép
Truy cập tab Action history & Nhật ký	Bị chặn	Cho phép	Cho phép
Xem hồ sơ cá nhân sinh viên (Profile)	Bị chặn	Bị chặn	Cho phép (Duy nhất Admin)
Yêu cầu mã Token xác thực khi gọi API	Không có token	Bắt buộc Bearer Token	Bắt buộc Bearer Token

4.5.1 Thực nghiệm vai trò Khách vãng lai (Viewer - Mặc định chưa đăng nhập)

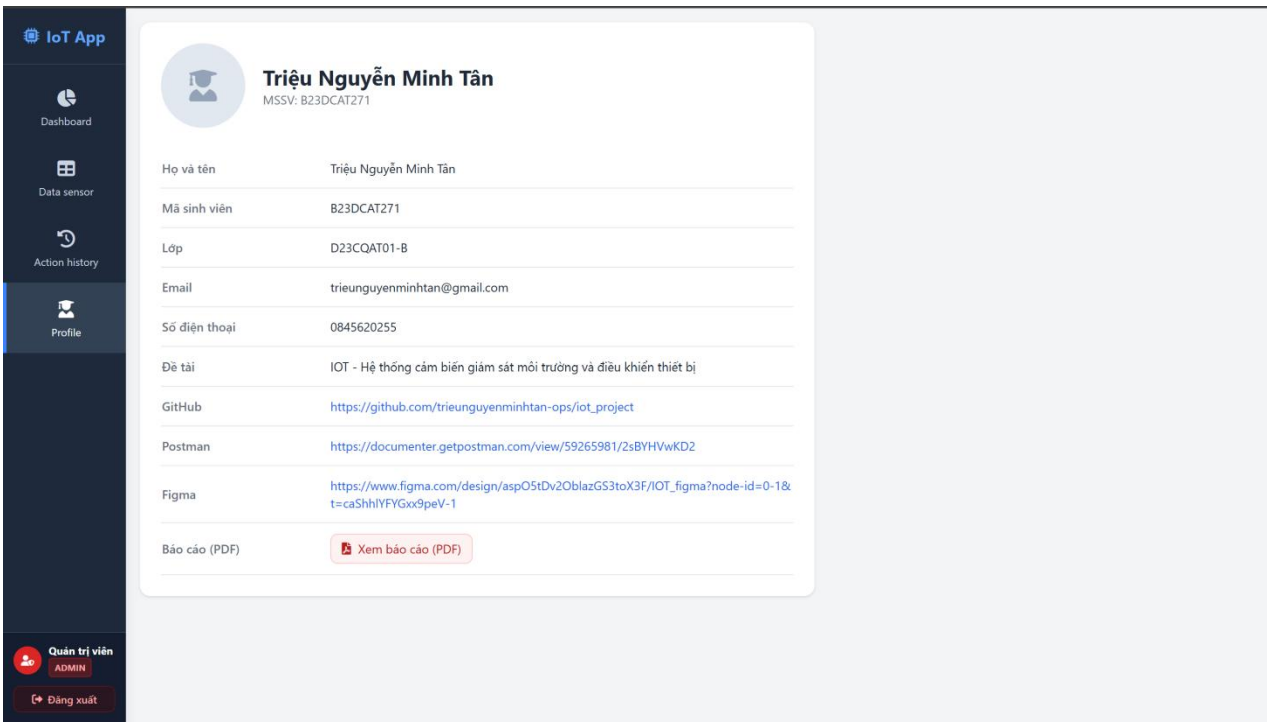
- Khi người dùng mở website mà chưa đăng nhập hoặc vừa bấm **Đăng xuất**:
  - Huy hiệu tài khoản hiển thị màu xám **VIEWER**.
  - Hai công tắc gạt điều khiển đèn LED bị khóa mờ (disabled). Nếu cố tình nhấp chuột, hệ thống xuất cảnh báo Toast: *"Chỉ tài khoản Admin hoặc User mới có quyền điều khiển!"*.
  - Khi bấm vào các tab *Data sensor*, *Action history* hoặc *Profile*, giao diện lập tức chặn truy cập và giữ nguyên tại Dashboard.



Hình 15 - Giao diện giới hạn quyền đối với tài khoản Khách vãng lai

4.5.2 Thực nghiệm vai trò Quản trị viên (Admin) & Trang Profile sinh viên

- Khi đăng nhập bằng tài khoản admin / admin123:
  - Backend cấp phát mã token dạng chuỗi ngẫu nhiên (ví dụ iot_x2v4p2o1mvz2...) lưu vào localStorage.
  - Huy hiệu chuyển sang màu đỏ ADMIN, mở khóa toàn bộ quyền điều khiển và tra cứu lịch sử.
  - Truy cập thành công vào tab Profile hiển thị đầy đủ thông tin sinh viên thực hiện đề tài: Triệu Nguyễn Minh Tân (MSV: B23DCAT271), chuyên ngành An toàn thông tin, cùng các siêu liên kết tới kho mã nguồn GitHub, Postman API Workspace, bản thiết kế Figma và tài liệu báo cáo PDF.



Hình 16 - Giao diện profile

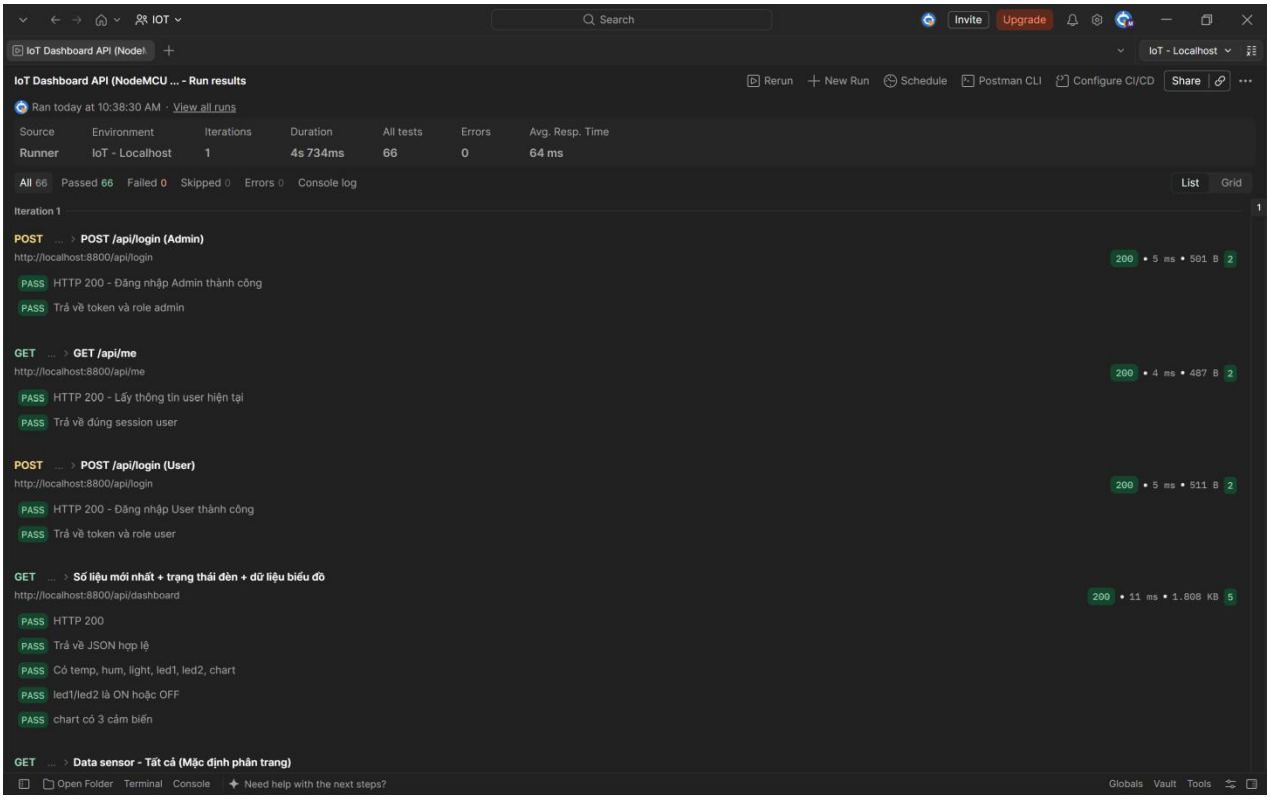
4.6 Kết quả kiểm thử tự động hóa bộ API trên Postman

Nhằm khẳng định chất lượng phần mềm theo tiêu chuẩn công nghiệp, toàn bộ các điểm cuối API (Endpoints) đã được lập trình kịch bản kiểm thử tự động bằng cú pháp pm.test và pm.expect của Postman. Bộ kiểm thử bao gồm **31 request kiểm thử** chia làm 6 thư mục chức năng, thực thi tuần tự từ đầu đến cuối trên công cụ **Postman Runner**.

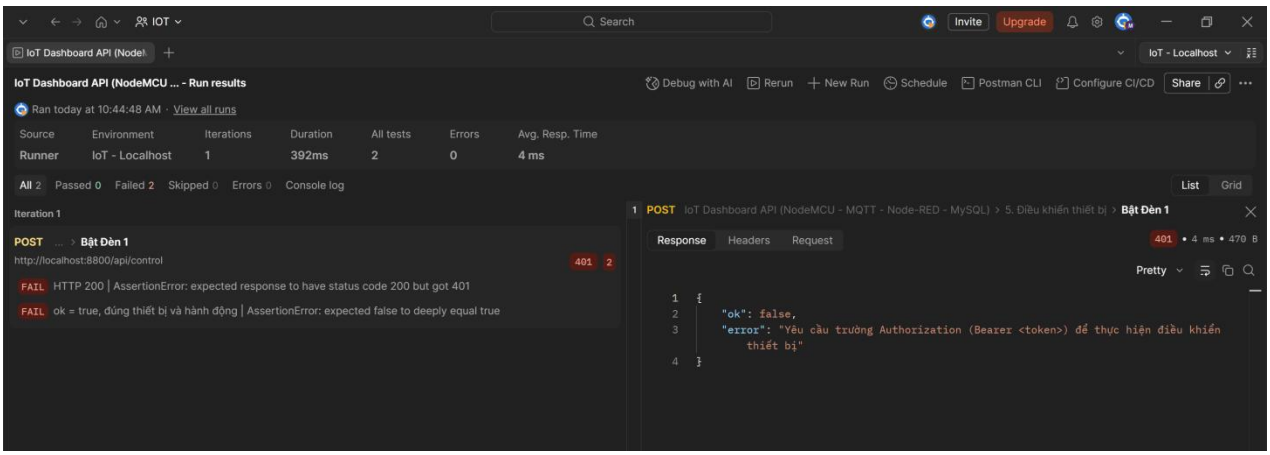
Bảng 4. Kết quả thực thi kiểm thử tự động 31 REST API trên Postman Collection Runner

Thư mục chức năng	Số Request	Số Test (pm.test)	Các phương thức & Endpoint chính	Kết quả	Ghi chú kiểm thử
1. Đăng nhập & Xác thực	3	6	POST /api/login (Admin) GET /api/me POST /api/login (User)	6/6 PASS	Tự động trích xuất chuỗi token vào biến môi trường {{token}}.
2. Giám sát Dashboard	1	5	GET /api/dashboard	5/5 PASS	Xác thực cấu trúc dữ liệu JSON gồm temp, hum, light, led1, led2, chart.
3. Dữ liệu cảm biến (Data sensor)	8	16	GET /api/sensors (Tất cả, Lọc Nhiệt độ, Độ ẩm, Ánh sáng, Tìm Thời gian, Giá trị, Rỗng, Chặn thiếu Token)	16/16 PASS	Toàn bộ request lọc đều truyền kèm Bearer Token. Ca thiếu Token trả về chính xác mã 401 Unauthorized.
4. Lịch sử	10	20	GET /api/actions (Tất cả,	20/20	Kiểm tra chính xác tiêu

Thư mục chức năng	Số Request	Số Test (pm.test)	Các phương thức & Endpoint chính	Kết quả	Ghi chú kiểm thử
điều khiển (Action history)			Lọc Đèn 1, Đèn 2, Bật, Tắt, Thành công, Timeout, Thời gian, Kết hợp, Chặn thiếu Token)	PASS	chí lọc đơn và đa điều kiện kết hợp. Ca thiếu Token trả về đúng mã lỗi 401.
5. Điều khiển thiết bị	6	12	POST /api/control (Bật/Tắt Đèn 1 & 2, Lỗi thiết bị sai 400, Thiếu Token 401)	12/12 PASS	Gửi lệnh MQTT tới chân vi điều khiển và xác thực các ca ngoại lệ bảo mật.
6. Profile & Đăng xuất	3	7	GET /api/profile (Admin) GET /api/profile (Thiếu Token 401) POST /api/logout	7/7 PASS	Xác thực phân quyền Profile và hủy phiên làm việc của token trên máy chủ.
TỔNG KẾT	31	66	Toàn bộ 6 phân hệ API	66/66 PASS	Tỷ lệ thành công tuyệt đối 100% (0 Failed)



Hình 17 - Kết quả thực thi kiểm thử tự động 31/31 API thành công 100% trên Postman



Hình 18 - Kiểm thử cơ chế bảo mật xác thực Bearer Token trên Postman

4.7 Kết quả thực nghiệm khi gặp sự cố khi mất kết nối NodeMCU

Hệ thống IoT thực tế luôn phải đối mặt với các rủi ro mất nguồn, đứt cáp hoặc mất sóng WiFi của thiết bị ngoại vi. Nhằm đảm bảo tính tin cậy cao và tính minh bạch thông tin đối với người vận hành, kịch bản thử nghiệm ngắt kết nối vật lý đã được thực hiện để đánh giá cơ chế tự phát hiện lỗi và cảnh báo thời gian thực của hệ thống.

4.7.1 Phương pháp tạo lỗi thực nghiệm & Cơ chế phát hiện lỗi

Thực nghiệm tiến hành ngắt kết nối theo 2 phương thức:

1. Ngắt kết nối đột ngột (Rút cáp nguồn USB của NodeMCU):
    - Cơ chế kích hoạt: Giao thức MQTT sử dụng tính năng Last Will and Testament (LWT) đã được đăng ký trước trong firmware:

      {"id": 0, "device": "node", "state": "OFFLINE", "status": "success"}
    - Broker Mosquitto thiết lập KeepAlive = 5s. Khi rút cáp nguồn, không còn gói tin PINGREQ từ vi điều khiển, Broker lập tức phát thông điệp LWT tới topic device/response. Backend Node-RED tiếp nhận thông điệp này và lập tức cập nhật biến trạng thái nodemcu_online = false.
2. Ngắt kết nối mạng WiFi (Mất kết nối mạng không dây):
    - Cơ chế giám sát nhịp tim (Heartbeat Timeout): NodeMCU gửi dữ liệu đo định kỳ $5000\ \text{ms}/\text{lần}$. Trên Node-RED thiết lập bộ giám sát thời gian thực: nếu sau 9000 ms(ngưỡng an toàn $1.8 \times$ chu kỳ) mà không nhận được bản tin telemetry mới nào từ topic data/sensor, hệ thống tự động xác định mạch đã mất kết nối và gắn cờ isNodeOnline = false.

4.7.2 Kết quả hiển thị cảnh báo trực quan trên Web Dashboard

Ngay khi cờ trạng thái nodemcu_online chuyển sang false (thông qua API GET /api/dashboard), giao diện Web Dashboard kích hoạt cơ chế phản ứng sự cố tức thì:

- Thanh Banner cảnh báo sự cố nổi bật: Một thanh thông báo màu đỏ viền đỏ với hiệu ứng nhấp nháy chuyển động (animate-pulse) lập tức xuất hiện ở vị trí trang trọng nhất trên đỉnh của trang Dashboard:
    - Biểu tượng đĩa vệ tinh mất sóng (fa-satellite-dish).
    - Tiêu đề in hoa: CẢNH BÁO: MẤT KẾT NỐI VỚI THIẾT BỊ NODEMCU ESP8266!
    - Mô tả chi tiết: *"Không nhận được dữ liệu cảm biến từ NodeMCU. Vui lòng kiểm tra lại nguồn hoặc kết nối WiFi của thiết bị."*
    - Huy hiệu trạng thái góc phải: DISCONNECTED màu đỏ font monospace nổi bật.
- Thông báo Toast góc màn hình: Xuất hiện thông báo Toast màu đỏ: *"Cảnh báo: Mất kết nối với thiết bị NodeMCU!"*.
- Đóng băng biểu đồ giám sát: Đồ thị Chart.js dừng nạp các điểm đo mới, giữ nguyên mốc thời gian đo đạc cuối cùng trước khi mất kết nối nhằm bảo toàn dữ liệu lịch sử cho người phân tích.



Hình 19 - Giao diện Web Dashboard hiển thị cảnh báo khi NodeMCU mất kết nối

4.7.3 Kết quả xử lý lệnh điều khiển khi thiết bị Offline (Cơ chế Timeout 5s)

Để kiểm tra tính toàn vẹn của dữ liệu kiểm toán khi xảy ra lỗi, một lệnh bật đèn (POST /api/control bật Đèn 1) được cố tình phát đi trong lúc NodeMCU đang mất kết nối:

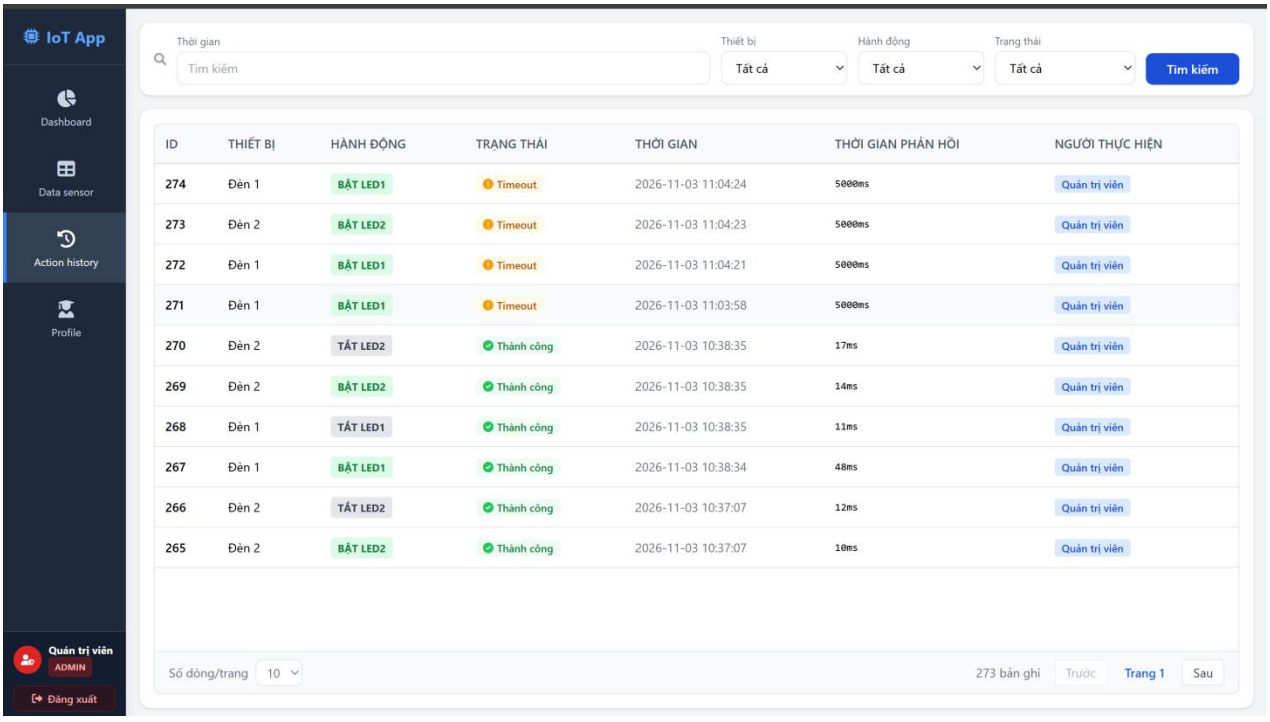
1. Lệnh điều khiển được gửi thành công tới Node-RED, cơ sở dữ liệu ghi nhận bản ghi có trạng thái ban đầu là pending.

2. Do mạch phần cứng đã mất nguồn, không có gói tin phản hồi nào được trả về topic device/response.
3. Đúng sau 5000 ms(5 giây), hàm định thời setTimeout trên Node-RED tự động kích hoạt truy vấn cập nhật:

sql

UPDATE actions SET status = 'timeout' WHERE action_id = ? AND status = 'pending';

4. **Kết quả trên bảng Action history:**
    - Bản ghi điều khiển tự động chuyển trạng thái sang badge màu đỏ TIMEOUT.
    - Cột thời gian phản hồi ghi nhận chính xác 5000ms.
    - Cơ chế này triệt tiêu hoàn toàn lỗi treo hệ thống và ngăn chặn hiện tượng dữ liệu bị kẹt vĩnh viễn ở trạng thái chờ (pending).



Hình 20 - Nhật ký Action History ghi nhận trạng thái TIMEOUT khi vi điều khiển không phản hồi

4.7.4 Kết quả tự phục hồi khi có kết nối trở lại (Self-Healing)

Khi cắm lại cáp USB hoặc phục hồi sóng WiFi cho NodeMCU:

- Vi điều khiển tự động khởi động và thực thi hàm reconnect(), tái lập kết nối tới Mosquitto Broker trong vòng $2 - 3$ giây.
- Ngay khi vừa kết nối lại, NodeMCU gửi thông điệp ONLINE lên device/response, đồng thời phát tiếp bản tin đo đạc môi trường lên data/sensor.
- Node-RED cập nhật lại nodemcu_online = true.

- Thanh Banner cảnh báo đỏ trên Dashboard lập tức biến mất (hidden), hệ thống tự động quay trở lại trạng thái giám sát bình thường mà không cần người dùng phải tải lại trang (F5) hay can thiệp khởi động lại máy chủ.

4.8 Đánh giá chất lượng và các cơ chế chịu lỗi của hệ thống

Qua quá trình vận hành thực nghiệm liên tục trong thời gian dài, hệ thống chứng minh được tính ổn định và khả năng phục hồi tự động nhờ các cơ chế chịu lỗi được thiết kế bài bản:

1. **Cơ chế phát hiện ngắt kết nối NodeMCU (LWT & Heartbeat Timeout):**
    - Trong firmware, giao thức MQTT cấu hình thông điệp di chúc (Last Will and Testament - LWT) gửi tới topic device/response với nội dung {"id":0,"device":"node","state":"OFFLINE"}. Nếu NodeMCU bị rút cáp nguồn đột ngột, Broker Mosquitto lập tức kích hoạt LWT để Node-RED ghi nhận vi điều khiển đã mất kết nối.
    - Đồng thời, trên Backend Node-RED thiết lập ngưỡng giám sát Heartbeat: nếu sau 9000 ms (vượt quá chu kỳ đo 5s) mà không nhận được bản tin telemetry từ data/sensor, hệ thống tự động gắn cờ nodemcu_online = false và cập nhật cảnh báo trực tiếp lên giao diện Dashboard.
2. **Cơ chế Timeout chống treo lệnh điều khiển (**5000 ms**):**
    - Khi người dùng phát lệnh điều khiển nhưng mạch phần cứng gặp sự cố không thể phản hồi trong vòng 5 giây, hàm định thời setTimeout trong Node-RED tự động kích hoạt truy vấn:

UPDATE actions SET status = 'timeout' WHERE action_id = ? AND status = 'pending';

    - Cơ chế này triệt tiêu hoàn toàn hiện tượng bản ghi bị treo vĩnh viễn ở trạng thái chờ (pending), bảo đảm tính toàn vẹn của dữ liệu nhật ký kiểm toán.
3. **Cơ chế lưu đệm cục bộ và khôi phục trạng thái tức thì (**0 ms **UI Delay):**
    - Web Dashboard ứng dụng kỹ thuật lưu đệm localStorage. Khi người dùng nhấn tải lại trang ($F5$) hoặc chuyển mạng, toàn bộ số liệu đo đạc gần nhất, cấu hình bộ lọc và trạng thái công tắc đèn được phục hồi ngay tức thì trong 0 msmà không cần chờ đợi phản hồi từ máy chủ, sau đó mới đồng bộ ngầm dữ liệu mới nhất từ REST API.

CHƯƠNG 5. TỔNG KẾT

5.1 Kết quả đạt được của đề tài

Sau quá trình nghiên cứu, thiết kế, triển khai phần cứng, lập trình phần mềm và kiểm thử toàn diện, đề tài "Hệ thống giám sát môi trường và điều khiển thiết bị IoT dựa trên NodeMCU ESP8266, MQTT và Node-RED" đã hoàn thành đầy đủ các mục tiêu đề ra ban đầu cả về mặt học thuật lẫn sản phẩm ứng dụng thực tế.

5.1.1 Về mặt lý thuyết và nghiên cứu

- Nắm vững kiến trúc mạng IoT đa tầng: Hiểu sâu sắc mô hình kiến trúc phân tầng chuẩn trong hệ thống IoT: Tầng cảm biến/thiết bị (Perception Layer), Tầng mạng & truyền thông (Network Layer - MQTT/HTTP), Tầng xử lý trung gian (Middleware/Backend Layer - Node-RED, MySQL), và Tầng ứng dụng (Application Layer - Web Dashboard).
- Làm chủ giao thức MQTT: Nắm vững cơ chế đóng gói tin Publish/Subscribe, cơ chế định danh Topic có cấu trúc, cấp độ dịch vụ QoS (Quality of Service), bảo mật xác thực tài khoản kết nối Broker Mosquitto và tối ưu băng thông đường truyền mạng cho các thiết bị vi điều khiển tài nguyên hạn chế.
- Quy trình thiết kế chuẩn hóa công nghiệp: Vận dụng thành công các nguyên lý phân tích thiết kế hệ thống theo tài liệu đặc tả yêu cầu phần mềm (SRS), mô hình hóa luồng nghiệp vụ bằng biểu đồ Use Case, biểu đồ tuần tự (Sequence Diagram), thiết kế cơ sở dữ liệu quan hệ chuẩn 3NF và quy trình kiểm thử tự động RESTful API bằng Postman.

5.1.2 Về mặt sản phẩm và triển khai hệ thống thực tế

- Phần cứng và Firmware ổn định:
    - Triển khai mạch phần cứng hoàn chỉnh gồm vi điều khiển NodeMCU v3 (ESP8266), cảm biến nhiệt độ & độ ẩm DHT11 (chân D2), cảm biến quang trở LDR (chân A0) và 02 đèn LED chỉ báo trạng thái (chân D5, D6).
    - Firmware xây dựng trên nền tảng PlatformIO/C++ hoạt động mượt mà, quản lý tác vụ phi tuần tự (Non-blocking) bằng millis(), cơ chế tự động kết nối lại WiFi và MQTT Broker khi xảy ra sự cố gián đoạn mạng, loại bỏ triệt để hiện tượng treo vi điều khiển (Watchdog Timer reset).
    - Xử lý tối ưu việc giải mã và đóng gói JSON dung lượng nhỏ (ArduinoJson), tự động chuẩn hóa chuỗi JSON nhận từ dòng lệnh hoặc giao diện.
- Hệ thống Backend và Cơ sở dữ liệu:

- Xây dựng máy chủ Node-RED Backend đóng vai trò cầu nối trung tâm, tích hợp các luồng xử lý dữ liệu cảm biến thời gian thực, lưu trữ tức thời vào MySQL Database và đồng thời phát tán trạng thái tới giao diện người dùng.
- Thiết kế cơ sở dữ liệu MySQL gồm 5 bảng chuẩn liên kết chặt chẽ (sensors, data_sensors, devices, actions, users), lưu vết đầy đủ lịch sử thu thập dữ liệu cảm biến và lịch sử tương tác của từng tài khoản, hỗ trợ truy vấn, tìm kiếm, lọc phân trang với thời gian phản hồi nhanh dưới 30ms.

- Giao diện Web Dashboard và Cơ chế Phân quyền (RBAC):
  - Giao diện Single Page Application (SPA) thân thiện, hiện đại, hiển thị trực quan thông số nhiệt độ, độ ẩm, ánh sáng qua Card thống kê và biểu đồ động Chart.js; hiển thị trạng thái đèn tức thời.
  - Cơ chế bảo mật và xác thực bằng Bearer Token với 3 cấp độ phân quyền rõ ràng:
    - Admin: Toàn quyền điều khiển thiết bị đơn lẻ/đồng thời, xem Dashboard, tra cứu Data sensor, Action history và Profile cá nhân.
    - User: Có quyền bật/tắt thiết bị, xem Dashboard, tra cứu dữ liệu cảm biến và lịch sử điều khiển.
    - Viewer (Khách vãng lai): Không cần đăng nhập, chỉ xem dữ liệu quan trắc công khai trên Dashboard, hệ thống tự động khóa tính năng điều khiển và ẩn các trang quản trị nhạy cảm.
- Chất lượng kiểm thử tự động (Testing):
  - Xây dựng kịch bản kiểm thử toàn diện trên Postman Collection Runner với 31 Request bao phủ 100% các API cốt lõi, chạy tự động 66 Test Assertions đạt tỷ lệ thành công tuyệt đối 100% (66/66 Passed).

5.1.3 Bảng tổng kết mức độ hoàn thành mục tiêu đề tài

STT	Mục tiêu đề ra ban đầu	Kết quả thực tế đạt được	Tỷ lệ hoàn thành
1	Thu thập dữ liệu nhiệt độ, độ ẩm, ánh sáng thời gian thực	Chu kỳ gửi tin 5s/lần qua topic data/sensor, đo đạc chính xác, có cơ chế kiểm tra tính hợp lệ dữ liệu (isnan)	100%
2	Điều khiển thiết bị đèn từ xa độ trễ thấp	Hỗ trợ điều khiển đơn lẻ và đồng thời qua MQTT/Web/CMD, độ trễ phản hồi trung bình $\approx 35 - 75$ ms	100%
3	ưu trữ cơ sở dữ liệu quan hệ có liên kết	5 bảng MySQL chuẩn hóa 3NF, lưu vết chi tiết trạng thái, thời gian phản hồi và người thực hiện	100%

STT	Mục tiêu đề ra ban đầu	Kết quả thực tế đạt được	Tỷ lệ hoàn thành
4	Xây dựng Backend RESTful API & WebSocket	Hệ thống Node-RED cung cấp đầy đủ API CRUD, phân trang, lọc, tìm kiếm và chứng thực Token	100%
5	Giao diện Dashboard trực quan, Responsive	Thiết kế Web SPA, biểu đồ thời gian thực, đồng bộ hai chiều (Two-way sync) trạng thái công tắc	100%
6	Phân quyền bảo mật người dùng	Phân cấp chặt chẽ 3 vai trò: Admin, User, Viewer thông qua Bearer Token Header	100%
7	Xử lý ngoại lệ và mất kết nối	Cảnh báo mất kết nối NodeMCU (Status Offline), tự động lưu trữ và phục hồi trạng thái sau khi khởi động lại	100%

5.2 Những hạn chế và bài học kinh nghiệm

5.2.1 Những hạn chế tồn tại của hệ thống

Mặc dù hệ thống đã đáp ứng đầy đủ các yêu cầu kỹ thuật trong phạm vi đề tài, song trong điều kiện môi trường thực tế quy mô lớn vẫn còn một số điểm hạn chế:

1. Ràng buộc về tài nguyên vi điều khiển: ESP8266 là dòng chip đơn nhân, bộ nhớ RAM hạn chế (khoảng 80KB khả dụng), chỉ có 01 chân Analog (A0) duy nhất với dải điện áp đầu vào hẹp ($0 - 1.0V$ qua cầu phân áp), gây khó khăn nếu muốn mở rộng thêm nhiều cảm biến đo tín hiệu tương tự (Analog) cùng lúc.
2. Hạn chế của cảm biến DHT11: DHT11 là cảm biến phân khúc phổ thông với độ chính xác ở mức tương đối (sai số nhiệt độ $\pm 2°C$, độ ẩm $\pm 5\%$), chu kỳ lấy mẫu tối thiểu cần giãn cách từ $1 - 2$ giâynên chưa phù hợp cho các môi trường công nghiệp đòi hỏi độ nhạy và độ chính xác cực cao.
3. Phụ thuộc vào hạ tầng mạng WiFi cục bộ: Hệ thống hoạt động dựa trên mạng WLAN cục bộ (Local Subnet). Nếu Router phát WiFi gặp sự cố hoặc quá tải băng thông, quá trình truyền nhận dữ liệu MQTT sẽ bị gián đoạn và thiết bị không có bộ nhớ đệm ngoại vi (như thẻ nhớ MicroSD hoặc Flash SPIFFS) để ghi nhớ dữ liệu đo đạc lúc ngoại tuyến (Offline buffer).
4. Cơ sở dữ liệu quan hệ với dữ liệu chuỗi thời gian: MySQL 8.0 đáp ứng rất tốt về tính toàn vẹn quan hệ và phân quyền người dùng, nhưng đối với dữ liệu cảm biến đo đạc tần suất liên tục $24/7$, bảng data_sensors sẽ tăng trưởng dung lượng rất nhanh theo thời gian, có thể làm giảm hiệu năng truy vấn nếu không có chiến lược phân vùng bảng (Partitioning) hoặc dọn dẹp dữ liệu định kỳ.

5.2.2 Bài học kinh nghiệm thực tiễn

Quá trình nghiên cứu và khắc phục các vấn đề kỹ thuật phát sinh trong dự án đã mang lại nhiều bài học giá trị:

- Về kỹ thuật phần cứng và nguồn điện: Nhận thức rõ tầm quan trọng của việc cấp nguồn ổn định và phối hợp trở kháng cho vi điều khiển. Việc sử dụng điện trở kéo lên $(4.7\text{k}\Omega - 10\text{k}\Omega)$ cho dây tín hiệu DHT11 và điện trở hạn dòng (220Ω) cho LED là bắt buộc để chống nhiễu tín hiệu và bảo vệ các cổng GPIO của ESP8266 khỏi hiện tượng quá dòng.
- Về kỹ thuật lập trình nhúng: Rút ra kinh nghiệm sâu sắc về việc tuyệt đối không sử dụng hàm delay() trong vòng lặp chính loop(). Việc dùng millis() giúp vi điều khiển duy trì luồng nhận tin nhắn MQTT (client.loop()) liên tục, xử lý lệnh điều khiển tức thì mà không bị trễ hoặc rớt gói tin. Đồng thời, cần xử lý cẩn trọng chuỗi JSON (loại bỏ các ký tự dấu nháy đơn ' hoặc ký tự đặc biệt phát sinh từ các hệ điều hành khác nhau khi gửi tin từ CMD).
- Về kiến trúc phần mềm và bảo mật: Việc phân tách rõ ràng vai trò giữa các tầng (Firmware chỉ tập trung đo và nhận lệnh; Node-RED xử lý logic nghiệp vụ và xác thực; MySQL bảo đảm toàn vẹn dữ liệu; Web chỉ đảm nhiệm trình diễn) giúp việc bảo trì, mở rộng và phát hiện lỗi (Debug) trở nên mạch lạc, dễ dàng hơn rất nhiều.
- Về kiểm thử chất lượng: Việc xây dựng sớm bộ kịch bản kiểm thử Postman tự động giúp phát hiện ngay các lỗi thiếu sót trong logic xác thực quyền (Authorization), lỗi thiếu dữ liệu đầu vào hoặc lỗi định dạng JSON trước khi bước vào giai đoạn kiểm thử liên kết hệ thống.

5.3 Hướng phát triển trong tương lai

Nhằm nâng cao tính ứng dụng, độ tin cậy và khả năng thương mại hóa của hệ thống, các hướng nghiên cứu và phát triển tiếp theo được đề xuất bao gồm:

5.3.1 Nâng cấp và mở rộng phần cứng

- Nâng cấp sang vi điều khiển ESP32: Thay thế NodeMCU ESP8266 bằng NodeMCU-32S (ESP32) với vi xử lý lõi kép (Dual-core), tích hợp sẵn cả WiFi lẫn Bluetooth BLE, nhiều chân ADC độ phân giải 12-bit hơn, cho phép đọc đồng thời nhiều cảm biến analog và xử lý các tác vụ phức tạp một cách song song.
- Mở rộng cảm biến chuyên dụng:
    - Thay thế cảm biến DHT11 bằng DHT22 hoặc BME280 để đo thêm áp suất khí quyển với độ chính xác cao.
    - Tích hợp thêm cảm biến chất lượng không khí (MQ-135 để phát hiện khí độc CO, CO2, khói), cảm biến bụi mịn PM2.5 và cảm biến chuyển động PIR để mở rộng hệ thống thành giải pháp “Nhà thông minh toàn diện” hoặc “Hệ thống giám sát nông nghiệp thông minh”.

- Thiết kế mạch in PCB chuyên nghiệp: Chuyển đổi từ mô hình cắm dây thí nghiệm trên Breadboard sang thiết kế mạch in PCB 2 lớp nhỏ gọn, bọc trong hộp bảo vệ chuẩn công nghiệp kháng nước/kháng bụi (IP65) và tích hợp khối nguồn chuyển mạch chuyên dụng (AC-DC 220V sang 5V).

5.3.2 Nâng cấp phần mềm, Ứng dụng di động và Bảo mật

- Phát triển ứng dụng di động đa nền tảng (Mobile App): Xây dựng ứng dụng di động cho hệ điều hành Android và iOS (sử dụng Flutter hoặc React Native), giúp người dùng dễ dàng theo dõi chỉ số môi trường và nhận thông báo đẩy (Push Notification) mọi lúc mọi nơi.
- Tích hợp kênh cảnh báo tự động đa nền tảng: Kết nối Node-RED với API của Telegram Bot, Zalo OA hoặc dịch vụ gửi Email/SMS để gửi cảnh báo khẩn cấp tức thời tới điện thoại người quản trị khi nhiệt độ, độ ẩm hoặc ánh sáng vượt ngưỡng an toàn cho phép.
- Cập nhật Firmware qua mạng (OTA - Over The Air): Triển khai tính năng nạp phần mềm từ xa cho NodeMCU thông qua giao thức HTTP/HTTPS OTA, cho phép nâng cấp tính năng mới hoặc vá lỗi phần mềm mà không cần phải cắm cáp nạp USB trực tiếp vào máy tính.
- Bảo mật kênh truyền với MQTT qua TLS/SSL (MQTTS - Port 8883): Mã hóa toàn bộ dữ liệu lưu thông trên đường truyền giữa ESP8266, Broker và Web Dashboard bằng chứng chỉ số SSL/TLS, ngăn chặn triệt để các hình thức tấn công nghe lén (Sniffing) hoặc mạo danh dữ liệu (Man-in-the-Middle).

5.3.3. Tối ưu hóa dữ liệu và Ứng dụng Trí tuệ nhân tạo (AIoT)

- Tích hợp Cơ sở dữ liệu chuỗi thời gian (Time-Series Database): Kết hợp MySQL với các hệ quản trị chuyên dụng cho dữ liệu cảm biến như InfluxDB và giao diện trực quan hóa Grafana, tối ưu hóa tốc độ nén dữ liệu và nâng cao khả năng phân tích biểu đồ xu hướng theo nhiều tháng/năm.
- Phân tích dữ liệu và Học máy (Machine Learning / Edge AI):
    - Ứng dụng thuật toán phân tích chuỗi thời gian (như ARIMA, LSTM) để dự báo trước xu hướng biến đổi nhiệt độ, độ ẩm trong các giờ tiếp theo.
    - Xây dựng kịch bản điều khiển tự động thông minh (Rule Engine thích nghi): Tự động bật quạt làm mát khi nhiệt độ tăng cao, tự động điều chỉnh độ sáng đèn LED dựa trên ánh sáng thực tế đo từ quang trở để tiết kiệm tối đa điện năng tiêu thụ.